

AI-Powered Diagnosis of Liver Disease Using Machine Learning

BITE497J – Project I

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology in Information Technology

By

Aditya Mittal

22BIT0645

Dhiraj Rajesh Motwani

22BIT0625

Prabal Dubey

22BIT0608

**Under the guidance of
Prof. Karthikeyan D**

**School of Computer Science Engineering and Information Systems
VIT, Vellore**



November, 2025

DECLARATION

We hereby declare that the BITE497J – Project – I thesis entitled “**AI-Powered Diagnosis of Liver Disease Using Machine Learning**” submitted by us, for the award of the degree of Bachelor of Technology in Information Technology, School of Computer Science Engineering and Information Systems to VIT is a record of Bonafide work carried out by us under the supervision of Dr. KARTHIKEYAN D, Assistant Professor Sr. Grade 1, SCORE, VIT, Vellore.

We further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidates

CERTIFICATE

This is to certify that the BITE497J – Project – I thesis entitled “**AI-Powered Diagnosis of Liver Disease Using Machine Learning**” submitted by **Aditya Mittal (22BIT0645), Dhiraj Rajesh Motwani (22BIT0625), and Prabal Dubey (22BIT0608)**, SCORE, VIT, for the award of the degree of Bachelor of Technology in Information Technology, School of Computer Science Engineering and Information Systems, is a record of Bonafide work carried out by them under my supervision during the period, 21.07.2025 to 30.11.2025, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfils the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the Guide

Internal Examiner

External Examiner

Head of the Department

Department of Information Technology

ACKNOWLEDGEMENT

It is our pleasure to express with a deep sense of gratitude to our BITE497J - Project – I guide **Dr. KARTHIKEYAN D**, Assistant Professor Sr. Grade 1, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for his constant guidance, continual encouragement, in our endeavour. Our association with him is not confined to academics only, but it is a great opportunity on our part to work with an intellectual and an expert in the field of AI and Machine Learning.

We would like to express our heartfelt gratitude to Honourable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Mr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**, Pro-Vice Chancellor **Dr. Partha Sharathi Mallick** and Registrar **Dr. Jayabarathi T**.

Our whole-hearted thanks to Dean **Dr. Daphne Lopez**, School of Computer Science Engineering and Information Systems, Head, Department of Information Technology, **Dr. Arivuselvan K**, Information Technology Project Coordinator **Dr. Suganya P**, SCORE School Project Coordinator **Dr. Thandeeswaran R**, all faculty, staff and members working as limbs of our university for their continuous guidance throughout our course of study in unlimited ways.

It is indeed a pleasure to thank our parents and friends who persuaded and encouraged us to take up and complete our project successfully. Last, but not least, we express our gratitude and appreciation to all those who have helped us directly or indirectly towards the successful completion of the project.

Place: Vellore

Date:

Aditya Mittal
Dhiraj Rajesh Motwani
Prabal Dubey

EXECUTIVE SUMMARY

This project presents an advanced AI-enabled web-based clinical decision support system designed to assist in the early identification of liver disease. Delayed diagnosis remains a significant contributor to liver-related morbidity and mortality, particularly in regions with limited access to specialized healthcare resources. To address this challenge, the system integrates machine learning-based prediction with real-time clinical data analysis to provide an accessible, fast, and intelligent screening solution.

The platform is developed using modern technologies including Streamlit for user interface, XGBoost for predictive modelling, Pandas and NumPy for data processing, and Google Gemini AI for intelligent recommendation generation. Users can input key demographic and laboratory parameters through an intuitive form interface, enabling the system to evaluate the likelihood of liver disease and instantly present risk classification results. The interface includes interactive data visualizations such as risk-level gauges, ranges of normal clinical values for reference, and AI-generated recommendations to support informed health decisions.

The underlying machine learning model demonstrates over 80% predictive accuracy, providing reliable preliminary screening outcomes. Alongside data-driven prediction, the integrated AI assistant supplies personalized lifestyle and medical suggestions for individuals deemed at elevated risk, emphasizing preventive care and informed decision-making.

This solution addresses several gaps in current diagnostic processes, including limited rural accessibility, reliance on invasive and high-cost diagnostic procedures, and lack of individualized guidance following screening results. By combining affordability, usability, and intelligent insights, it represents a step toward democratized healthcare technology and reinforces the importance of early intervention in chronic disease management.

Future development plans include integrating mobile application support for broader accessibility, interoperability with hospital systems via real-time medical data APIs, and expansion of the platform to support screening for multiple chronic diseases. Overall, this initiative contributes to enhancing proactive healthcare practices by empowering individuals and healthcare providers with rapid, data-driven liver disease risk evaluation and personalized health guidance.

TABLE OF CONTENTS

Chapter	Title	Page
CHAPTER 1: INTRODUCTION		
1.1	Background of Liver Disease Problem	1
1.2	Need for AI-Powered Diagnosis	2
1.3	Introduction to the LiverSage	3
1.4	Problem Statement	5
1.5	Objectives	5
1.5.1	Primary Objective	5
1.5.2	SMART Objectives (SO1–SO5)	5
1.6	Scope and Limitations	6
1.7	Organization of the Report	7
CHAPTER 2: LITERATURE REVIEW		
2.1	Global Liver Disease Statistics and Impact	8
2.2	Contributors to Delayed Diagnosis	9
2.3	Existing Diagnostic Solutions	10
2.3.1	Traditional Methods (LFTs, Biopsies)	10
2.3.2	AI-Based Systems (Random Forest, Neural Nets)	10
2.4	Machine Learning in Healthcare	11
2.4.1	XGBoost, Random Forest	11

Chapter	Title	Page
2.4.2	Role of Gemini AI	11
2.5	Gaps in Current Solutions	12
2.5.1	Functionality Gaps	12
2.5.2	Accessibility and Usability Gaps	12
2.5.3	Personalization Gaps	13
CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE		
3.1	System Overview	15
3.2	Architecture Diagram	16
3.3	Technology Stack	16
3.3.1	Frontend: Streamlit	17
3.3.2	Backend: Python with XGBoost	17
3.3.3	Model Storage: File-Based Deployment	17
3.3.4	AI Layer: Google Gemini	17
3.4	Data Flow Diagram	18
CHAPTER 4: DETAILED DESIGN AND METHODOLOGY		
4.1	Agile Development Methodology	20
4.1.1	Sprint Planning and Execution	20

Chapter	Title	Page
4.2	UI/UX Design	23
4.2.1	Wireframes	23
4.2.2	Key Screens (Input Form, Dashboard, Results)	23
4.3	Feature Design	24
4.3.1	Standard Clinical Reference Range Display	24
4.3.2	Real Time Input Validation	25
4.3.3	Prediction History	25
4.3.4	AI Model Inference Engine	25
4.3.5	Lifestyle and Diet Recommendation Engine	26
CHAPTER 5: IMPLEMENTATION		
5.1	Development Environment	27
5.2	Frontend Implementation	28
5.2.1	Page Configuration and Layout	28
5.2.2	Custom Styling	29
5.2.3	Input Form with Validation	29
5.2.4	Risk Assessment Dashboard	30
5.3	Backend Implementation	30
5.3.1	Model and Data Loading	31
5.3.2	Prediction Pipeline	31

Chapter	Title	Page
5.3.3	Gemini AI Integration	31
5.4	Deployment	32
5.4.1	Local Streamlit Server Deployment	32
5.4.2	Cloud Deployment (Planned)	32
5.5	Code Snippets (Key Modules)	33
CHAPTER 6: TESTING AND RESULTS		
6.1	Testing Strategy	35
6.2	Performance Metrics	36
6.2.1	Model Performance	36
6.2.2	System Response Time	37
6.3	Effectiveness Analysis	37
6.3.1	Cross-Validation Results	37
6.3.2	Feature Importance	37
6.4	User Flow Validation	38
CHAPTER 7: PROJECT MANAGEMENT		
7.1	Timeline	40
7.2	Weekly Progress Log	41
7.3	Resources and Tools	46
7.4	Risk Assessment and Mitigation	47

Chapter	Title	Page
CHAPTER 8: SUMMARY, LIMITATIONS, AND FUTURE WORK		
8.1	Summary of Achievements	51
8.2	Current Limitations	51
8.3	Future Enhancements	52
8.4	Conclusion	53
CHAPTER 9: REFERENCES		54
APPENDICES		56

LIST OF FIGURES

Figure No.	Title	Page
Fig. 1.1	Global Liver Disease Statistics by Region (Bar Chart)	2
Fig. 1.2	LiverSage System Overview & User Flow Diagram	4
Fig. 2.1	Rising Trend of Liver Diseases	9
Fig. 2.2	Evolution of AI in Medical Diagnostics (Timeline)	11
Fig. 3.1.1	High-Level System Architecture	15
Fig. 3.2	LiverSage System Architecture Diagram	16
Fig. 3.4	Data Flow	21
Fig. 4.1	Agile Development Workflow for LiverSage	22
Fig. 6.2.1	Model Performace Metrics	36

LIST OF TABLES

Table No.	Title	Page
Table 3.3	Technology Stack Overview	17
Table 6.2.1	Model Accuracy Metrics	36
Table 6.2.2	Response Time	37
Table 6.4	User Flow Validation	38
Table 7.1	Project Phase Timeline	40
Table 7.2	Weekly Progress log	41
Table 7.3	Resource Utilization	46
Table 7.4	Risk Register and Mitigation	48

List of Abbreviations

AI – Artificial Intelligence
ALT – Alanine Aminotransferase (SGPT)
AST – Aspartate Aminotransferase (SGOT)
ALP – Alkaline Phosphatase
APIs – Application Programming Interfaces
CV – Cross-Validation
CSV – Comma-Separated Values
EDA – Exploratory Data Analysis
EHR – Electronic Health Records
ILPD – Indian Liver Patient Dataset
LFT – Liver Function Test
ML – Machine Learning
NAFLD – Non-Alcoholic Fatty Liver Disease
RF – Random Forest
SGPT – Serum Glutamic-Pyruvic Transaminase (ALT)
SGOT – Serum Glutamic-Oxaloacetic Transaminase (AST)
UI – User Interface
UX – User Experience
WHO – World Health Organization
XAI – Explainable Artificial Intelligence
XGB / XGBoost – Extreme Gradient Boosting

CHAPTER 1: INTRODUCTION

Liver disease is a major global public health challenge, contributing significantly to illness and death worldwide. As one of the body's most essential organs, the liver supports crucial functions including detoxification, metabolism, protein synthesis, and regulation of vital biochemical processes. However, conditions such as hepatitis, cirrhosis, fatty liver disease, and liver cancer often develop silently, with minimal symptoms in early stages. This delayed presentation makes timely diagnosis difficult, leading to severe complications and preventable deaths each year.

Conventional diagnostic procedures, including liver function tests (LFTs), medical imaging, and specialist evaluation, remain the foundation of liver disease detection. While reliable, these methods pose certain limitations: they can be expensive and invasive, require skilled medical professionals, and are often inaccessible in rural or resource-limited regions. Moreover, manual clinical interpretation may introduce delays or human-error-based variations in diagnosis.

Advances in digital healthcare and the growing availability of patient data have opened new possibilities for AI-driven medical diagnostics. Machine learning (ML) algorithms can process large volumes of clinical data, identify subtle disease patterns, and deliver predictions with high accuracy and speed. By analyzing key factors like bilirubin levels, enzyme activities, protein concentrations, age, and demographic variables, these models can assist in early identification of liver disorders without invasive procedures.

This project introduces a Liver Disease Prediction System that leverages machine learning to promote early detection and decision support. The solution includes an intuitive web-based Flask application for real-time prediction, supports modular updates for future datasets and models, and aims to integrate intelligent features such as personalized recommendations via APIs like Gemini. By combining medical expertise with automated intelligence, this system seeks to strengthen clinical workflows, enhance healthcare accessibility, and improve patient outcomes through proactive and data-driven care.

1.1 Background of Liver Disease Problem

Liver diseases have become a major global public health concern, ranking among the leading causes of illness and premature death. The liver plays a vital role in metabolism, detoxification, and maintaining overall biochemical balance in the body. When its function is compromised, the consequences can be severe and life-threatening. Despite their seriousness, liver diseases

often develop silently, with minimal or non-specific symptoms in early stages. As a result, many patients are diagnosed only when the condition has progressed significantly, reducing the effectiveness of treatment options.

Global health reports indicate that millions of people die each year due to liver-related conditions such as fatty liver disease, hepatitis, cirrhosis, and liver cancer. Many of these cases could be prevented through early detection and timely medical intervention. The rising burden of lifestyle factors, metabolic disorders, and limited access to diagnostic facilities in under-served regions further highlights the need for improved and accessible liver disease screening methods.

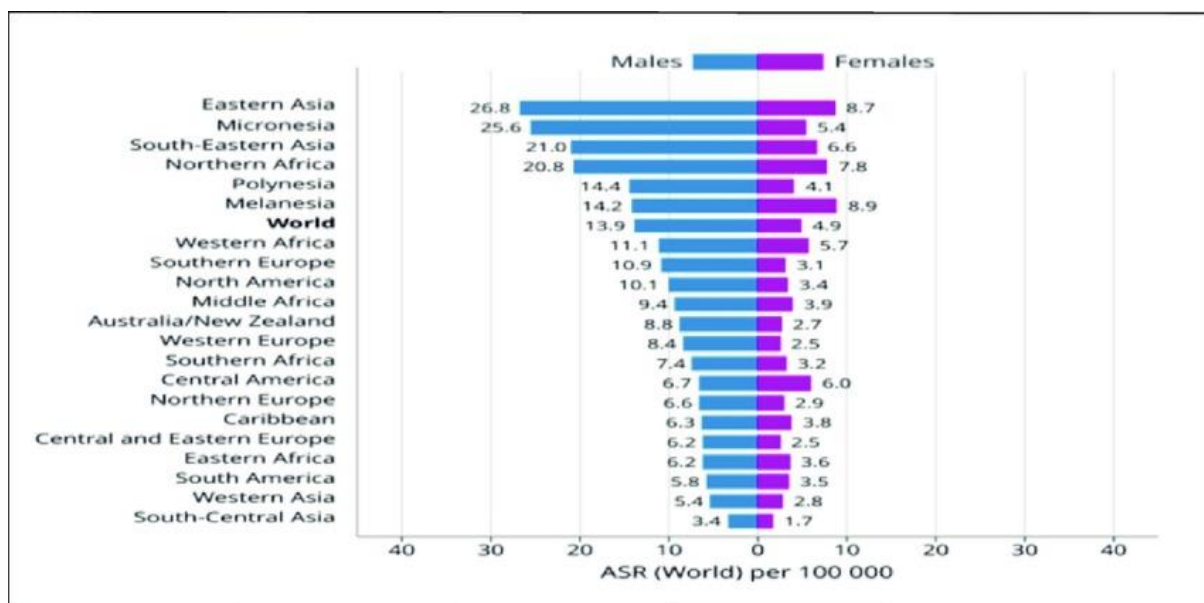


Fig. 1.1 Global Liver Disease Statistics by Region (Bar Chart)

1.2 Need for AI-Powered Diagnosis

Traditional liver disease diagnosis primarily depends on liver function tests (LFTs), imaging scans, and expert clinical judgment. While these techniques are reliable, they can be invasive, expensive, and not always easily accessible, particularly in rural or resource-limited healthcare settings. Additionally, manual interpretation of diagnostic reports may introduce subjectivity and delays, increasing the chances of late detection and complications.

With the increasing digital transformation in healthcare, artificial intelligence (AI) and machine learning (ML) have emerged as powerful tools to support clinical decision-making. AI-powered diagnostic systems can analyze large volumes of patient data, identify subtle patterns, and predict disease risk with high precision and efficiency. These systems enable faster

assessment, reduce dependency on specialized medical personnel, and offer scalable, cost-effective screening solutions. By enhancing early detection capabilities and improving diagnostic accuracy, AI-driven tools play a crucial role in bridging healthcare gaps and advancing proactive liver disease management.

1.3 Introduction to LiverSage

This system represents an advanced AI-driven platform designed to assist in the early detection and assessment of liver disease risk. It leverages machine learning techniques, particularly the XGBoost algorithm, to analyze key clinical and biochemical parameters such as age, gender, bilirubin levels, enzyme activity, and protein indicators. By processing these inputs, the system generates an immediate risk prediction, helping users and healthcare practitioners identify potential liver abnormalities at an early stage.

The application is built with a streamlined and interactive interface using Streamlit, enabling effortless data entry and intuitive result visualization. Additionally, the system is integrated with Google Gemini to provide personalized health suggestions based on the prediction outcome. This combination of predictive analytics and AI-generated guidance moves beyond traditional diagnostic checks, offering users not only a risk evaluation but also tailored lifestyle and monitoring recommendations. Overall, the system aims to enhance awareness, support preventive healthcare, and facilitate timely intervention.

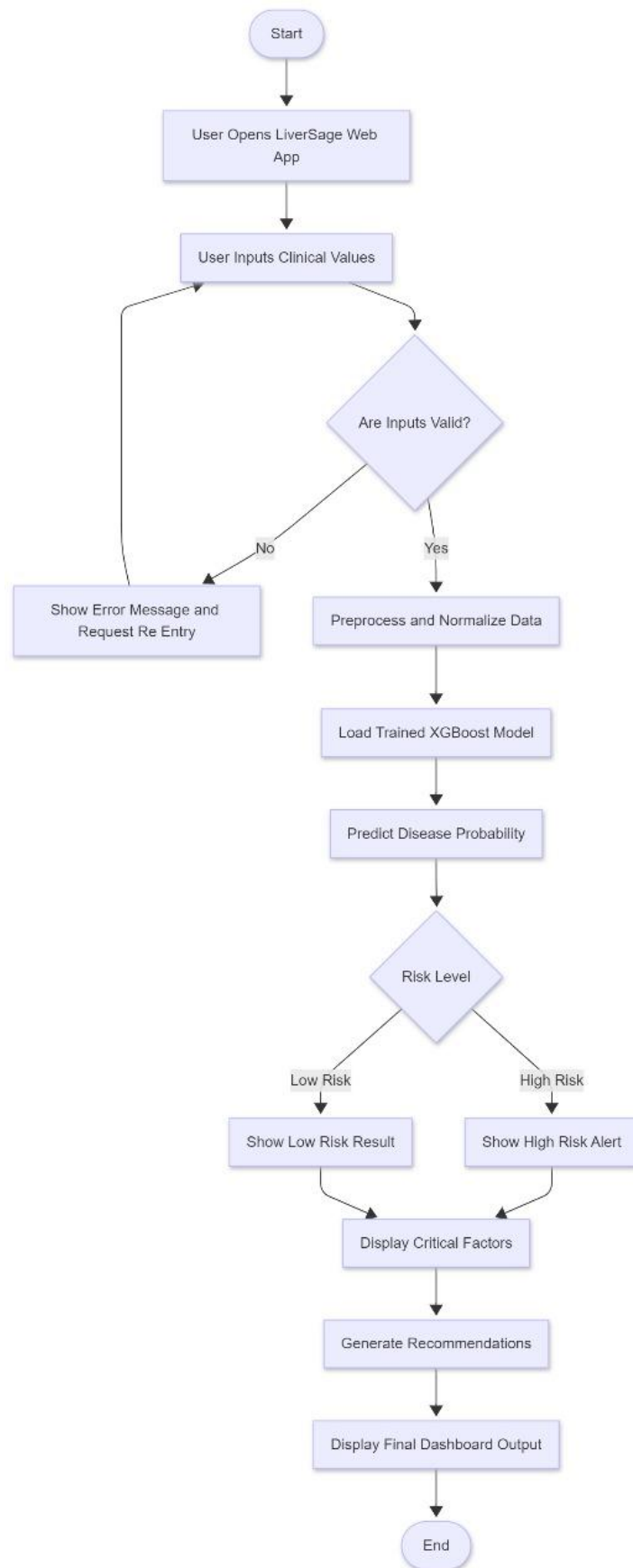


Fig. 1.2 LiverSage System Overview & User Flow Diagram

1.4 Problem Statement

Early identification of liver disease remains a significant challenge in modern healthcare. Many liver conditions advance silently, displaying minimal or non-specific symptoms until serious damage has already occurred. This frequently leads to delayed diagnosis, reducing treatment effectiveness and increasing mortality risk. Conventional diagnostic procedures rely on laboratory tests, imaging, and specialist consultation, which may be invasive, time-consuming, and costly. Furthermore, manual evaluation of clinical results introduces a possibility of human error and inconsistent interpretation.

Accessibility to specialized hepatology care and advanced diagnostic facilities is often limited in rural and resource-constrained regions, preventing timely screening and guidance. Existing tools for liver disease prediction are largely research-based, lacking real-time functionality, scalability, and patient-centric design. Additionally, most current systems do not provide tailored health recommendations or continuous monitoring support. Therefore, there is a need for an intelligent, automated, and user-friendly solution that enhances early detection, expands accessibility, and delivers personalized assistance for liver disease management.

1.5 Objectives

1.5.1 Primary Objective

The primary objective of this project is to develop an intelligent, AI-powered Liver Disease Prediction System capable of facilitating early and accurate detection of liver disorders. By leveraging advanced machine learning algorithms, particularly XGBoost, the system aims to analyze key biochemical and demographic indicators to predict liver disease risk with high precision. The platform is designed to be intuitive and accessible, using Streamlit to provide a seamless and interactive user experience for both medical practitioners and general users. The ultimate goal is to create a reliable tool that assists in timely disease identification, promotes preventive healthcare, and enhances clinical decision-making through data-driven insights.

1.5.2 SMART Objectives (SO1–SO5)

- **SO1: Collect and preprocess clinical data**

Gather a structured dataset containing demographic details and biochemical parameters relevant to liver health. Perform data cleaning, normalization, and feature engineering to ensure high-quality input for model training and accurate prediction outcomes.

- **SO2: Train models like XGBoost and evaluate performance**

Develop and train machine learning models, with a primary focus on XGBoost, to classify liver disease risk. Evaluate model performance using metrics such as accuracy, precision, recall, and F1-score to ensure reliability and optimize prediction capability.

- **SO3: Deploy via Streamlit for real-time predictions**

Build and deploy a user-friendly web interface using Streamlit that enables users to input lab results and receive instant prediction results, ensuring accessibility and efficient real-time interaction with the system.

- **SO4: Integrate Gemini for personalized recommendations**

Incorporate Google Gemini AI to generate personalized health guidance, lifestyle recommendations, and clinical suggestions based on the user's prediction results, enhancing user engagement and decision-making support.

- **SO5: Extend to other diseases in future**

Design the system with scalability in mind, allowing expansion to include prediction modules for additional diseases, thereby increasing its scope and impact in the healthcare domain.

1.6 Scope and Limitations

Scope:

This project focuses on the domain of healthcare analytics, specifically targeting the early prediction of liver disease using machine learning techniques. The system analyzes clinically relevant parameters and applies an optimized XGBoost model to assess the likelihood of liver abnormalities. Furthermore, a user-friendly Streamlit interface enables real-time interaction, making the platform suitable for academic, research, and preliminary clinical decision-support demonstrations. The system is also integrated with Gemini-based recommendations to provide users with personalized lifestyle and preventive health insights, thereby enhancing the overall utility and user experience.

Limitations:

Despite its practical value, the system is intended primarily for educational and prototype purposes and should not be treated as a replacement for professional medical diagnosis or treatment. The dataset used for model training represents static and historical records; hence,

real-time clinical data streams and continuous patient monitoring are not incorporated. Additionally, model accuracy is dependent on the quality and diversity of the dataset, and performance may vary across broader demographic populations. External clinical validation and regulatory clearance would be required before the system can be deployed as a certified medical diagnostic tool in real-world healthcare environments.

1.7 Organization of the Report

- **Chapter 2 – Literature Review:** Provides an in-depth review of existing research on liver disease prediction, machine learning in healthcare, and related diagnostic systems.
- **Chapter 3 – System Architecture:** Describes the overall architecture of the proposed Liver Disease Prediction System, detailing the components, data flow, and integration of ML, Streamlit, and Gemini modules.
- **Chapter 4 – Design Methodology:** Explains the project design strategy including dataset preprocessing, feature selection, model building, tuning, and evaluation methods.
- **Chapter 5 – Implementation:** Covers the practical development process, code execution steps, technology stack, and deployment through Streamlit.
- **Chapter 6 – Testing and Results:** Discusses system testing strategies, performance metrics (accuracy, classification reports, confusion matrix), and analysis of model results.
- **Chapter 7 – Project Management:** Highlights planning, task distribution, timelines, resource management, and development workflow.
- **Chapter 8 – Conclusion and Future Work:** Summarizes project outcomes, key findings, and future enhancements such as model improvements and multi-disease prediction capabilities.
- **Chapter 9 – References:** Lists all academic sources, research papers, datasets, and supporting literature used in the project.

CHAPTER 2: LITERATURE REVIEW

The literature review highlights a clear trend toward machine-learning-based approaches for liver disease prediction, moving beyond conventional diagnostic methods. Early studies focused on simple models such as Logistic Regression, Decision Trees, and SVM. While these provided moderate results, recent research shows that ensemble models like Random Forest, Gradient Boosting, and particularly XGBoost offer significantly higher accuracy and reliability. A recurring theme across studies is the importance of proper feature selection and preprocessing techniques, such as correlation-based filtering and dimensionality reduction, which improve model performance by removing irrelevant or noisy features. However, inconsistencies in dataset preparation and feature engineering make it difficult to directly compare outcomes across studies. Another major limitation in existing research is the reliance on small and region-specific datasets — most notably the Indian Liver Patient Dataset (ILPD) — which limits model generalization to wider populations. Additionally, limited focus on model interpretability and deployment challenges provides motivation for developing a scalable, explainable, and user-friendly system like LiverSage.

2.1 Global Liver Disease Statistics and Impact

Liver disease represents a significant global health burden and continues to rank among the leading causes of morbidity and mortality worldwide. According to reports from the World Health Organization (WHO), millions of individuals succumb to liver-related conditions each year, including hepatitis, cirrhosis, alcoholic liver disease, and non-alcoholic fatty liver disease (NAFLD). The prevalence of liver disorders is increasing due to rising rates of obesity, diabetes, alcohol consumption, and viral infections, particularly in low- and middle-income countries where access to preventive care and diagnostic facilities is limited.

Additionally, liver diseases often progress silently and remain undetected until they reach advanced stages, resulting in late diagnoses and reduced treatment success rates. Beyond mortality, liver disease imposes a substantial economic and social impact through hospitalizations, long-term treatments, loss of productivity, and diminished quality of life. Therefore, early detection, awareness, and technologically-driven screening mechanisms have become critical in addressing this growing healthcare challenge.

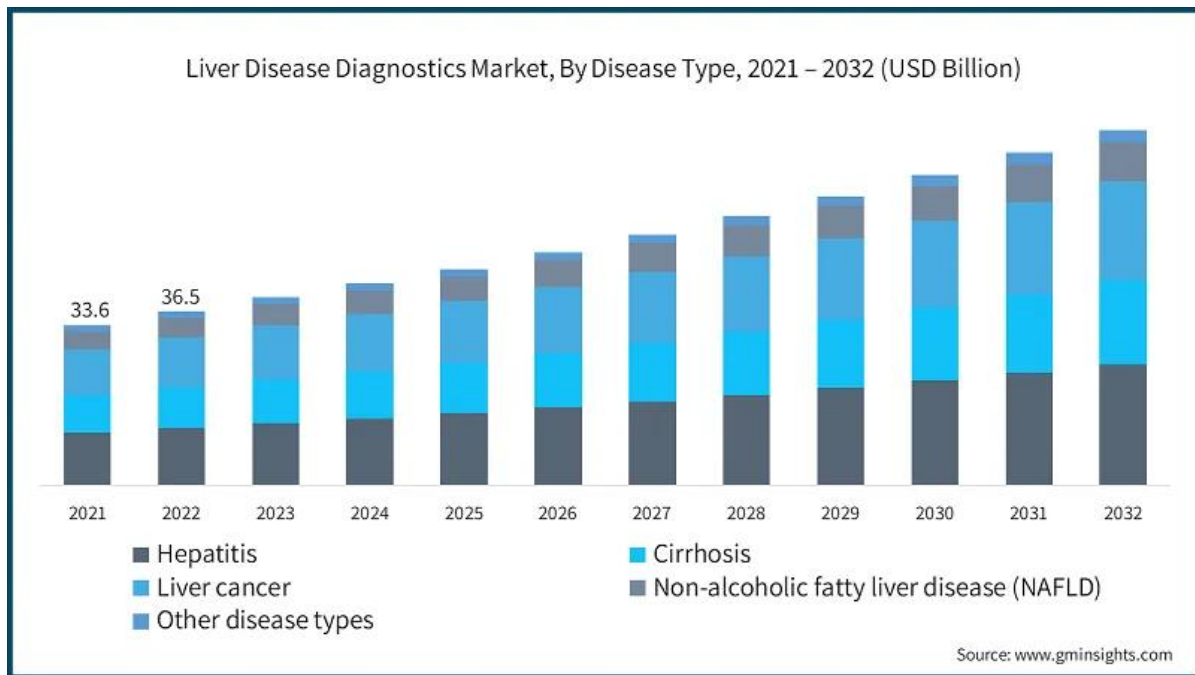


Fig. 2.1 Rising Trend of Liver Diseases

2.2 Contributors to Delayed Diagnosis

The delayed diagnosis of liver disease remains a critical challenge in global healthcare, primarily due to the asymptomatic nature of its early stages. Many patients do not exhibit noticeable symptoms until significant liver damage has occurred, resulting in late medical intervention and reduced treatment effectiveness. Furthermore, conventional diagnostic techniques often rely on clinical expertise and subjective interpretation of biochemical markers and imaging results, which may lead to inconsistencies and potential oversight.

Limited access to specialized diagnostic facilities, particularly in rural and resource-constrained regions, further contributes to delayed detection. High costs associated with advanced diagnostic procedures and the lack of routine screening programs also discourage timely evaluation. Additionally, variability in patient awareness and insufficient preventive health practices play a role in postponing medical assessment. Collectively, these factors underline the need for automated, accessible, and objective diagnostic solutions to facilitate early and reliable identification of liver disease.

2.3 Existing Diagnostic Solutions

2.3.1 Traditional Methods (LFTs, Biopsies)

Traditional diagnostic methods for liver disease primarily include Liver Function Tests (LFTs) and liver biopsies. LFTs measure biochemical markers such as bilirubin, albumin, and liver enzymes to evaluate liver health. While widely used, these tests can sometimes yield inconclusive results and require expert clinical interpretation, which may introduce subjectivity. Additionally, LFTs alone are insufficient to accurately distinguish between different types and severities of liver disorders.

Liver biopsy, considered the gold standard for definitive diagnosis, involves extracting liver tissue for microscopic examination. Although highly accurate, biopsies are invasive, painful, and carry risks such as bleeding and infection. They are also costly and not feasible for routine screening or large-scale implementation, especially in low-resource settings. Consequently, the limitations associated with traditional diagnostic procedures underscore the need for alternative, non-invasive, and cost-effective techniques for early liver disease detection.

2.3.2 AI-Based Systems (Random Forest, Neural Nets)

Recent advancements in artificial intelligence have paved the way for AI-based systems in liver disease prediction, demonstrating significant improvements over conventional diagnostic techniques. Studies have shown that machine learning models, particularly ensemble algorithms such as Random Forest and Gradient Boosting, achieve high accuracy and robustness by combining multiple decision trees and leveraging feature importance. These models effectively capture complex relationships between clinical variables, offering enhanced predictive performance.

Neural network-based approaches have also been explored, utilizing layered architectures to learn non-linear patterns in medical datasets. While neural networks can deliver strong results, they often require large datasets and substantial computational resources, which may limit their applicability in resource-constrained settings.

Overall, AI-driven diagnostic frameworks provide faster, scalable, and more consistent prediction capabilities. However, challenges such as data quality, interpretability, and deployment constraints highlight the ongoing need for balanced, efficient, and clinically integrable models in liver disease prediction.

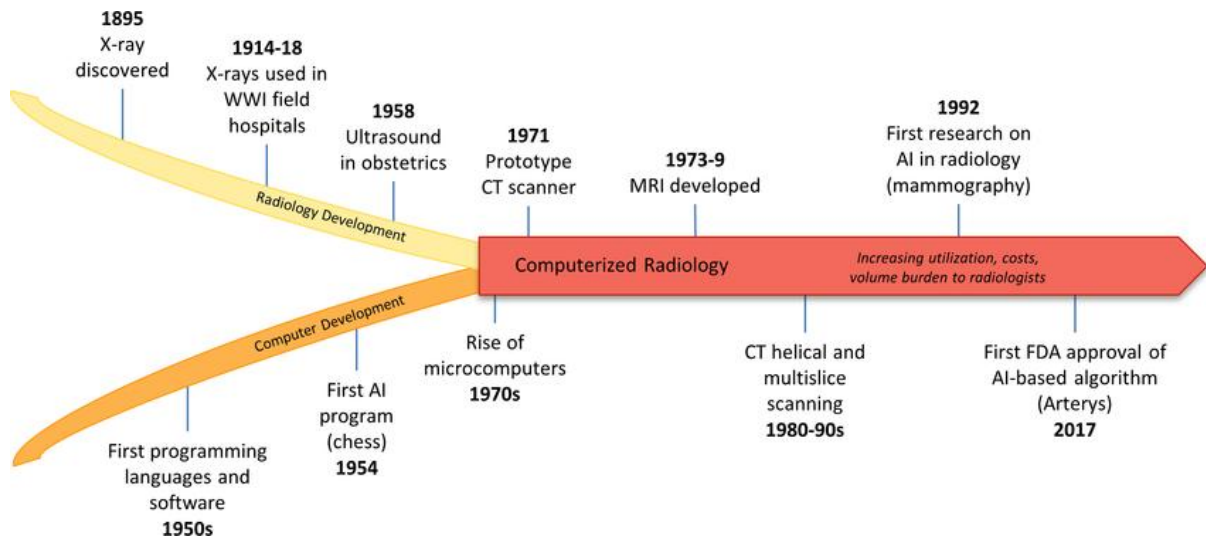


Fig. 2.2 Evolution of AI in Medical Diagnostics

2.4 Machine Learning in Healthcare

2.4.1 XGBoost, Random Forest

XGBoost and Random Forest are widely recognized as two of the most effective machine learning algorithms for clinical prediction tasks, including liver disease detection. These models are favored due to their ability to handle complex, non-linear data relationships and deliver consistently high accuracy across diverse datasets. Random Forest leverages multiple decision trees to reduce overfitting and improve generalization, making it particularly reliable for medical data that may contain noise or variability.

XGBoost, an optimized gradient boosting framework, enhances predictive performance through regularization, advanced tree-pruning techniques, and efficient computation. It is well-suited for structured clinical datasets and has demonstrated superior accuracy and faster training compared to traditional boosting algorithms.

Together, these algorithms provide strong classification capability, robust handling of imbalanced data, and enhanced interpretability through feature importance analysis, making them ideal choices for developing reliable AI-driven liver disease prediction systems.

2.4.2 Role of Gemini AI

Gemini AI plays a pivotal role in enhancing the overall functionality of the liver disease prediction system by providing personalized, actionable recommendations based on user inputs and prediction outcomes. While the machine learning model identifies the likelihood of liver

disease, Gemini extends the system's capability by offering tailored lifestyle guidance, dietary suggestions, preventive measures, and wellness tips to support healthier liver function.

By analyzing key parameters and user-specific health indicators, Gemini delivers context-aware insights such as optimal nutritional habits, recommended exercise routines, hydration practices, and avoidance of risk-enhancing behaviors like alcohol consumption or high-fat diets. This ensures the system does not merely provide a prediction but also empowers users with practical steps to improve their health and potentially manage early-stage liver conditions.

Through this personalized recommendation framework, Gemini AI bridges the gap between prediction and real-world health management, making the platform more holistic, user-centric, and clinically relevant.

2.5 Gaps in Current Solutions

2.5.1 Functional Gaps

- **Gap 1:** Most existing systems focus only on model prediction and lack integrated platforms combining ML prediction with lifestyle guidance and follow-up support.
- **Gap 2:** Current AI tools in liver diagnostics do not offer explainability or interactive recommendations tailored to patient health indicators such as bilirubin, ALT/AST levels, or albumin.

LiverSage Solution: LiverSage integrates XGBoost prediction, Streamlit-based clinical interface, and Gemini-powered personalized recommendations including diet, lifestyle changes, and preventive steps. Future explainability modules (SHAP) enhance clinical trust and model transparency.

2.5.2 Accessibility and Usability Gaps

- **Gap 3:** Most liver AI systems are research-based and not deployed on accessible web interfaces for real-time use by general users or primary clinics.
- **Gap 4:** Diagnostic AI platforms often require medical software installations or specialist handling, limiting reach in rural and low-resource settings.

LiverSage Solution: A lightweight, web-based Streamlit deployment enables users to access predictions without hospital-grade equipment. The system supports simple form-based inputs and is optimized for general healthcare usability and accessibility.

2.5.3 Personalization & Clinical Support Gaps

- **Gap 5:** Existing ML liver prediction models provide binary results (disease/no disease) without personalized dietary, lifestyle, or follow-up recommendations.
- **Gap 6:** Minimal focus on patient education and preventive awareness to support long-term health improvement.

LiverSage Solution: Gemini-assisted intelligent counselling suggests liver-friendly diets, alcohol management guidance, fitness habits, hydration planning, and periodic test reminders. This bridges the gap between diagnosis and preventive care.

CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

The system architecture is designed to provide a scalable, reliable, and user-friendly platform for AI-assisted liver disease prediction and clinical decision support. This chapter outlines the complete architectural design, focusing on modular data processing, real-time machine learning inference, and seamless user interaction through a web-based interface. The system follows a layered pipeline integrating user input, preprocessing logic, predictive modeling, and recommendation-generation, ensuring efficient and secure handling of sensitive health data.

The architecture follows a client-model-server structure, where the frontend interface enables users to input clinical values, while the AI engine processes data to deliver prediction results and health insights. Streamlit serves as the primary interaction layer, supporting real-time visualizations and clear diagnostic feedback. A dedicated machine learning layer hosts the trained model, allowing fast and accurate inference using optimized algorithms such as XGBoost.

Emphasis is placed on modularity and scalability to accommodate future enhancements — including API-based integration for hospital systems, wearable health data ingestion, Gemini-powered lifestyle and dietary recommendations, and multi-disease prediction expansion. The layered design ensures that clinical data collection, preprocessing, prediction, and personalized feedback work in a smooth and secure pipeline, supporting proactive and data-driven healthcare delivery.

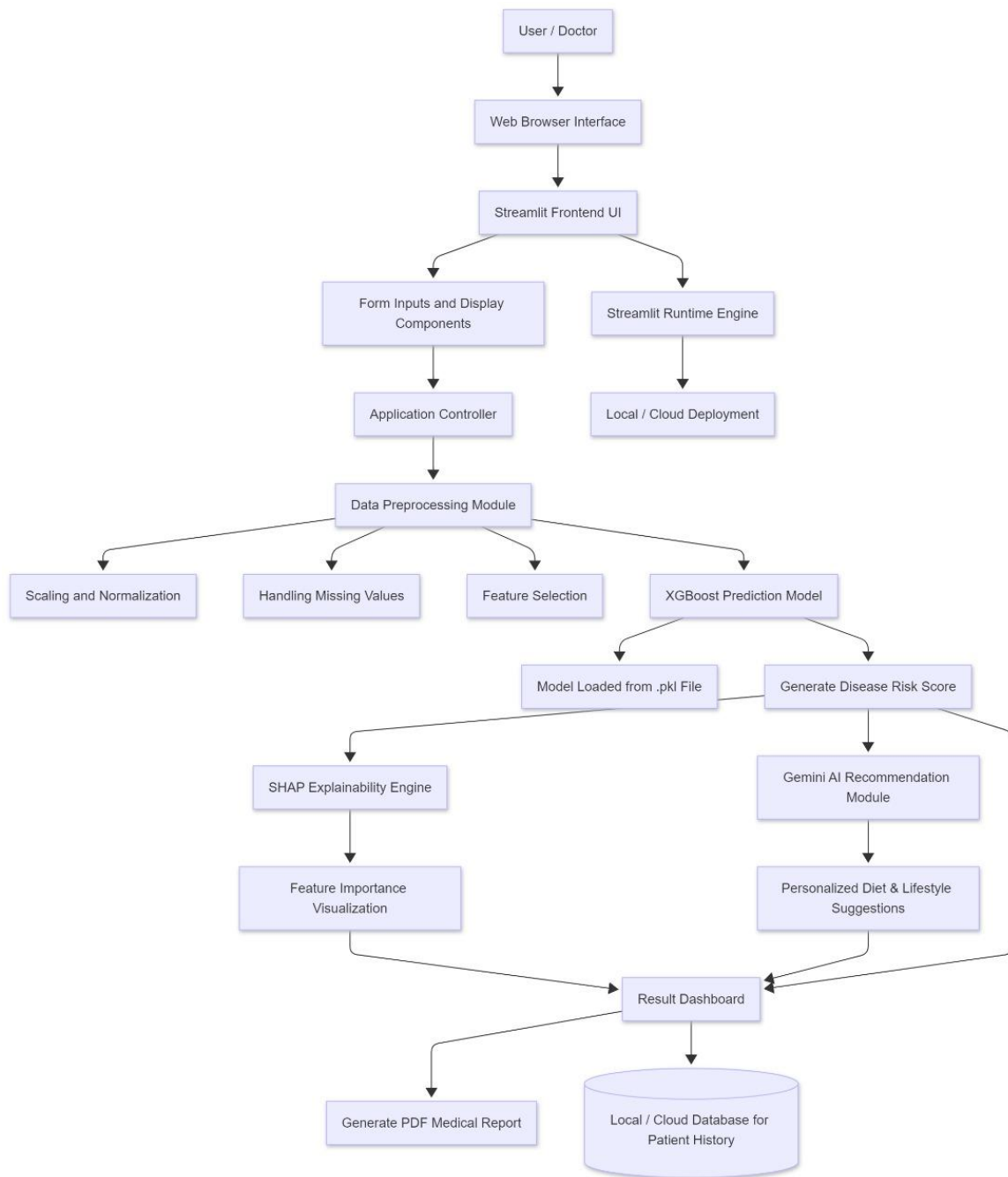


Fig. 3.1.1 High-Level System Architecture

3.1 System Overview

The system operates as an AI-enabled clinical decision support platform with four primary layers:

1. User Interface Layer – Streamlit Web Application

Handles user interaction and input of clinical parameters such as bilirubin, enzyme levels, albumin, age, and gender. Provides an intuitive dashboard for prediction results and visual insights.

2. Preprocessing & Processing Layer – Data Validation and Feature Engineering

Standardizes user inputs, encodes categorical values, scales numeric fields, and ensures consistency with model-training formats. Manages data quality steps including input validation, handling missing values, and format normalization.

3. Machine Learning Layer – Trained Model Execution (XGBoost / Random Forest)

Hosts serialized machine learning models loaded via Pickle/Joblib. Executes real-time prediction on processed input data and generates disease-risk outputs along with probability scores for interpretability.

4. Analytics & Recommendation Layer – Visualization and AI Insights

Displays prediction results using charts/gauges and generates personalized lifestyle and diet recommendations via integration with Gemini AI. Provides interpretation support and health guidance.

The system runs as a web-based solution deployable on local environments or cloud platforms (e.g., Streamlit Cloud/AWS), ensuring accessibility and real-time performance for users and healthcare providers.

3.2 Architecture Diagram

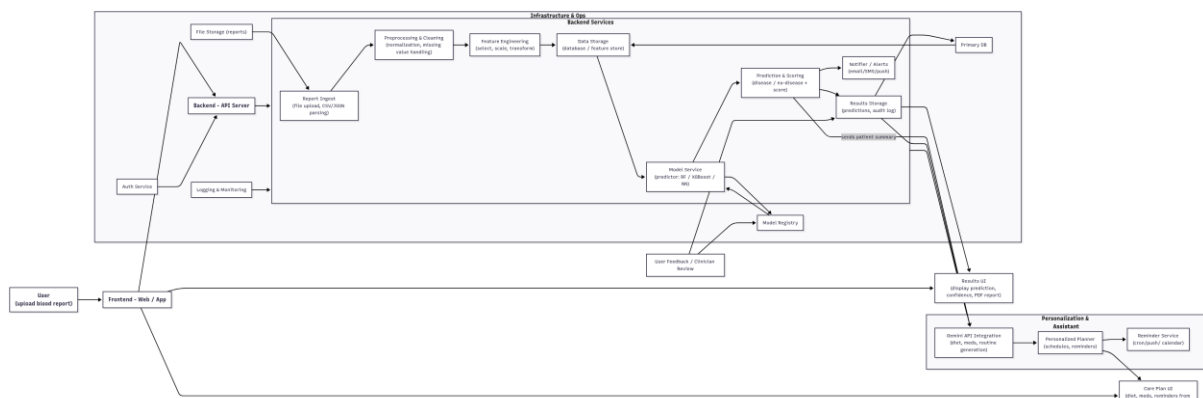


Fig. 3.2 LiverSage System Architecture Diagram

3.3 Technology Stack

The system utilizes a lightweight yet powerful technology stack optimized for real-time clinical input processing, machine learning-based prediction, and AI-driven personalized recommendations.

3.3.1 Frontend: Streamlit

The user interface is developed using Streamlit, enabling an interactive and responsive web application. Streamlit provides form-based input fields for entering patient clinical values (such as bilirubin, ALT, AST, albumin, and total proteins) and displays prediction results along with visual indicators. Its rapid prototyping capability ensures a seamless user experience without complex frontend development overhead.

3.3.2 Backend: Python with XGBoost

Python serves as the backend processing layer, handling data preprocessing, feature encoding, and coordination with the trained machine learning model. XGBoost is used as the primary prediction algorithm due to its high accuracy, robustness, and efficiency in handling structured healthcare datasets. Python libraries such as Pandas and NumPy support efficient data handling and computation.

3.3.3 Model Storage: File-Based Deployment

The trained liver disease prediction model is serialized and stored locally using Pickle/Joblib format (.pkl file). During execution, the system loads this pre-trained model for real-time inference. This file-based deployment eliminates the need for a live database and ensures fast, offline-capable prediction functionality.

3.3.4 AI Layer: Google Gemini

Google Gemini is integrated to enhance the system with personalized health guidance. After generating the prediction, Gemini provides tailored recommendations based on patient-specific parameters, including lifestyle improvements, diet suggestions, hydration routines, and early-intervention preventive tips. This layer enriches the system by offering meaningful, actionable health advice beyond numerical prediction results.

Table 3.3 Technology Stack Overview

Layer	Technology Used	Purpose / Function
Frontend	Streamlit	User interface for input, visual results, chart rendering

Layer	Technology Used	Purpose / Function
Backend / ML Engine	Python, XGBoost	Liver disease prediction & decision logic
Model Storage	.pkl File (Serialized Model)	Stores trained liver prediction model (no DB required)
AI Recommendation Layer	Google Gemini API	Provides personalized lifestyle, diet, and health suggestions
Data Processing	Pandas, NumPy	Dataset cleaning, preprocessing, and feature handling
Visualization	Matplotlib / Streamlit Charts	Plots risk levels, trends, and model insights
Security	Streamlit Session / Input Validation	Ensures safe input handling & secure model execution
Deployment	Local System / Streamlit Cloud	Hosting and running LiverSage application
Testing & Validation	Medical dataset testing, manual validation	Model accuracy checks & output verification

3.4 Data Flow Diagram

The data flow begins when the user uploads a liver function report:

1. **User uploads report** → Client UI (Streamlit) → uploads CSV/form with patient details.
2. **Prediction request** → POST /predict → Flask API receives file + metadata.
3. **File preprocessing** → API sends file to Preprocessor → parse & clean data → extract required features.

4. **Model load & inference** → Preprocessed features → ModelRunner loads liver_model.pkl → executes predict() → returns output (Liver Disease / No Disease + probability scores).
5. **Prediction response delivery** → Flask API returns JSON result to frontend → UI displays disease risk classification & confidence scores.
6. **Personalized plan request (optional)** → If user requests recommendations → POST /personalized-plan → features passed to Gemini API.
7. **AI health guidance** → Gemini generates lifestyle and dietary guidance → returned to UI with structured plan.

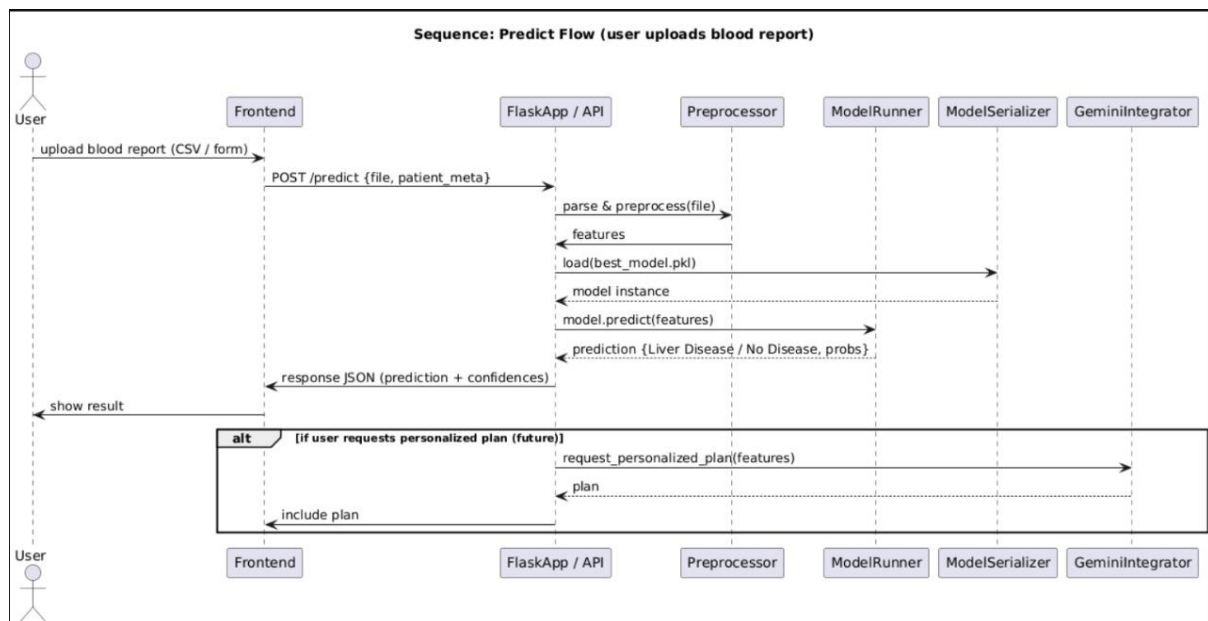


Fig. 3.4 Data Flow

CHAPTER 4: DETAILED DESIGN AND METHODOLOGY

The proposed liver disease prediction system follows a structured methodology to ensure accuracy and practical usability. First, clinical and biochemical patient data are collected from verified medical sources, including key attributes like bilirubin levels, liver enzymes, albumin, and proteins. The data undergo preprocessing through cleaning, normalization, feature selection, and balancing techniques such as SMOTE. Multiple machine learning models, including Random Forest, XGBoost, and Artificial Neural Networks, are trained using an 80:20 split and validated with k-fold cross-validation. Model performance is assessed through accuracy, precision, recall, F1-score, and confusion matrix analysis, and the best-performing model is selected. The chosen model is serialized and deployed in a Flask-based web application for real-time predictions. An intuitive interface allows clinicians and users to input medical values and receive instant results. Future enhancements include integration with Gemini API for personalized diet and lifestyle recommendations, wearable device connectivity, and expansion to additional chronic disease prediction.

4.1 Agile Development Methodology

4.1.1 Sprint Planning and Execution

The LiverSage project followed the Agile Scrum methodology to ensure structured development, iterative improvement, and seamless adaptation to evolving requirements. The overall development timeline was divided into **four sprints**, each lasting **two weeks**, with clearly defined objectives, deliverables, and evaluation checkpoints. Regular communication cycles—including **daily stand-ups**, **weekly review meetings**, and **retrospective sessions**—helped monitor progress, address challenges, and refine the workflow.

Sprint Breakdown

- **Sprint 1 – Data Pipeline Setup & EDA**
 - Sourcing and consolidating liver dataset.
 - Performing data cleaning, normalization, and missing-value handling.
 - Conducting exploratory data analysis (EDA) to understand feature patterns and disease correlations.
 - Producing initial visualizations and statistical insights.

- **Sprint 2 – Model Development & Optimization**
 - Implementing baseline machine learning algorithms.
 - Tuning hyperparameters for Random Forest and XGBoost models.
 - Evaluating models using accuracy, precision, recall, and F1-score metrics.
 - Selecting the best performing model for deployment.
- **Sprint 3 – Application Development**
 - Building Flask-based backend services for prediction.
 - Designing a user-friendly interface using HTML/CSS (and Streamlit for advanced interactive version).
 - Integrating model prediction pipeline with web input/output flow.
- **Sprint 4 – Testing, Deployment & Documentation**
 - Performing system testing (unit and functional testing).
 - Enhancing UI responsiveness and improving prediction latency.
 - Preparing user documentation and technical project reports.
 - Conducting final sprint review and incorporating feedback from the project guide.

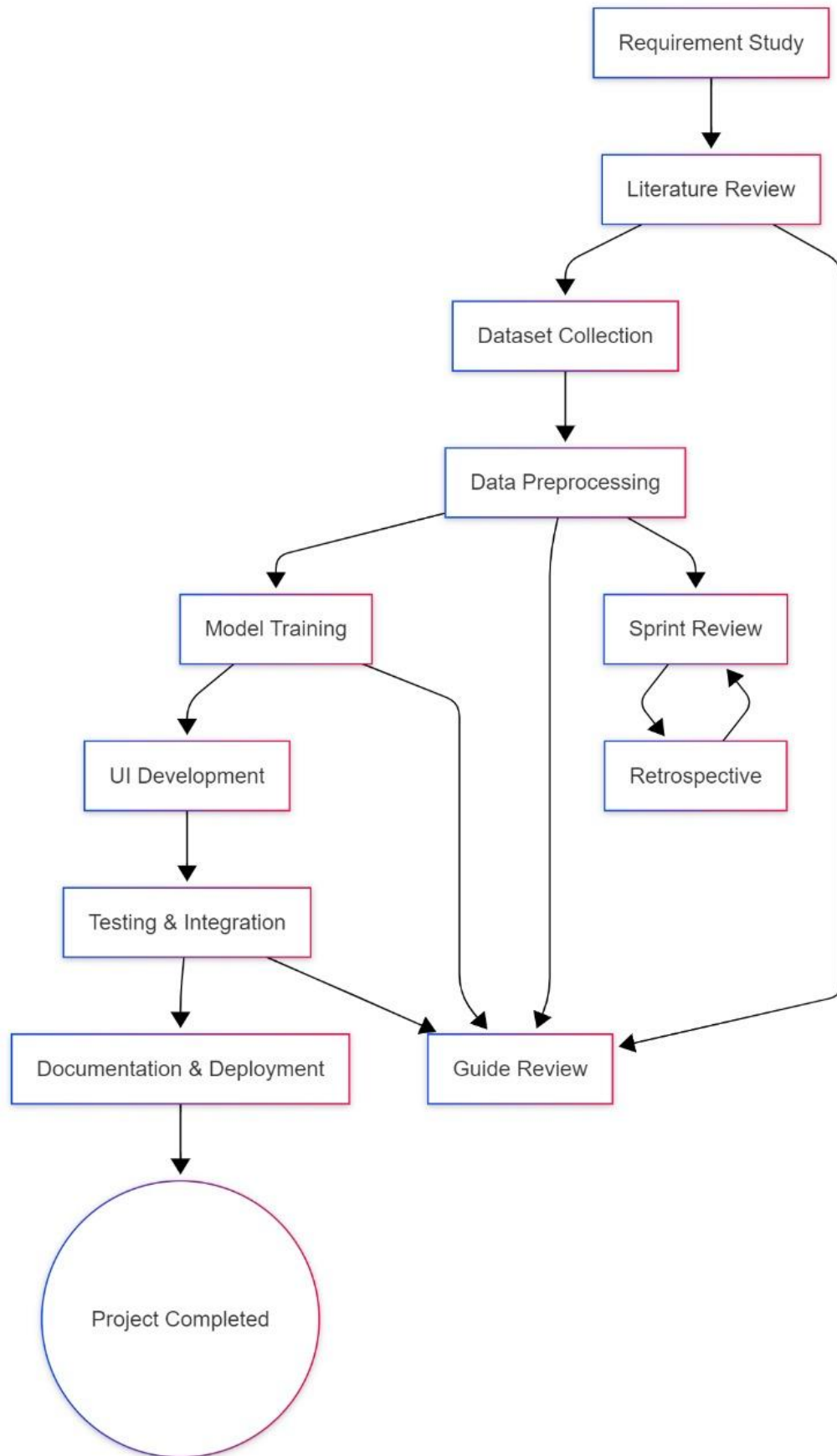


Fig. 4.1 Agile Development Workflow for LiverSage

4.2 UI/UX Design

The user interface of LiverSage was designed with a strong focus on **simplicity, clinical relevance, and accessibility**. The objective was to create a system that allows both healthcare professionals and patients to enter clinical data effortlessly and receive AI-driven liver disease predictions in under **10 seconds**. Emphasis was placed on clean layouts, medical-grade readability, and intuitive navigation to minimize cognitive load and enhance usability in real healthcare settings.

4.2.1 Figma Wireframes

High-fidelity wireframes were created using **Figma** to conceptualize key user interactions and ensure a consistent design language. The layouts were tested against usability principles such as clarity, contrast, and input efficiency.

Design Components:

- **Input Form:** Structured input fields for biochemical values (e.g., bilirubin, ALT, AST, albumin) with clearly labeled units and placeholder guidance.
- **Result Page:** Displays classification output—**Healthy / Liver Disease**—along with probability confidence levels and a visual **risk meter**.
- **Patient History Tab:** Provides access to previous predictions for authenticated users, enabling longitudinal tracking and clinical reference.

Color Palette:

- **Medical Blue (#007BFF)** — evokes trust, healthcare reliability, and clarity
- **White background** — enhances contrast, cleanliness, and readability

4.2.2 Key Screens

The final UI includes multiple functional screens to support end-to-end prediction and user workflow:

- **Dashboard**
 - Displays recent predictions and user activity insights
 - Includes model accuracy badge and quick-access buttons to prediction form

- **Prediction Input Form**
 - Real-time input validation for medical ranges
 - Tooltips explaining clinical parameters (e.g., ALT, AST significance)
 - Submission triggers instant backend model inference
- **Result Screen**
 - Displays **diagnosis**, confidence percentage, and **color-coded risk gauge**
 - Option to download a **PDF report** for clinical records or consultation
 - Recommendation placeholders for future AI-generated guidance (diet/lifestyle)

4.3 Feature Design

The LiverSage platform incorporates essential diagnostic and usability-focused features to assist both healthcare professionals and patients in liver health assessment. Each feature has been designed to ensure clinical relevance, accuracy, data security, and a seamless user experience.

4.3.1 Standard Clinical Reference Ranges Display

LiverSage includes an integrated reference panel displaying standard clinical value ranges for all liver function parameters. This feature enables users to compare their input values against medically accepted thresholds and gain contextual understanding of their health status even before prediction.

Key functionalities:

- Displays normal reference ranges for key liver biomarkers such as Total Bilirubin, Direct Bilirubin, AST, ALT, ALP, Albumin, and Total Protein.
- Provides tooltips and brief medical explanations for each parameter (e.g., “High Bilirubin may indicate jaundice or liver inflammation”).
- Color-coded indicators highlight whether entered values fall within normal or abnormal ranges.
- Helps users interpret blood report values intuitively, supporting informed decision-making.

- Enhances educational value for patients and caregivers by promoting health awareness and early-stage self-monitoring.

4.3.2 Real-Time Input Validation

To maintain data integrity and accuracy, the system includes front-end validation mechanisms.

Features:

- Input constraints ensure valid ranges for clinical parameters e.g., Age (1–120 years), Total Bilirubin (0.1–80 mg/dL), ALT/AST levels.
- Instant feedback prevents incorrect submissions and eliminates erroneous values.
- Improves model reliability by guaranteeing **clean and standardized inputs**.
- Uses JavaScript for real-time validation and error prompts.

4.3.3 Prediction History

LiverSage maintains a comprehensive record of previous predictions for users who opt to create accounts.

Key capabilities:

- Structured table displaying past diagnostic outcomes with timestamp and values entered.
- Enables longitudinal liver health tracking for chronic patients.
- Records can be exported as **CSV or PDF** for clinical use, hospital submissions, or doctor consultations.
- Enhances transparency and supports **medical record archiving**.

4.3.4 AI Model Inference Engine

The platform integrates a high-performance machine learning engine to deliver accurate predictions.

Key specifications:

- Backend loads a **pre-trained XGBoost model** serialized in .pkl format.
- Accepts ten clinical inputs to generate both:

- **Binary classification** → Healthy / Liver Disease
- **Prediction confidence score** → probability output
- Optimized for real-time inference with minimal latency, ensuring smooth patient experience.

4.3.5 Lifestyle & Diet Recommendation Engine

The system provides supportive recommendations for users seeking to improve liver health outcomes.

Features:

- Generates AI-based lifestyle suggestions after prediction (e.g., low-fat diet, alcohol restriction).
- Personalized diet guidelines based on biomarkers (e.g., high bilirubin → low-fat, high-fiber diet).
- Includes hydration, exercise, and sleep recommendations.
- Helps in preventive care and long-term patient wellness.
- Future enhancement: Integrate Gemini for clinically-aware suggestion refinement.

CHAPTER 5: IMPLEMENTATION

The implementation of LiverSage converts the system design into a fully functional, deployable clinical decision-support application. This chapter documents the development environment, frontend and backend structure, machine-learning integration, model serialization and loading, visualization components, testing strategy, and deployment workflow. The application is implemented using modern Python tooling and best practices: a Streamlit-based frontend for rapid interactive UI development; a Python backend that handles preprocessing, model inference (XGBoost), and API endpoints; Pandas and NumPy for data processing; and SHAP for explainability. The trained model is serialized with Joblib/Pickle and loaded at runtime for low-latency prediction. Google Gemini is integrated as an AI recommendation layer to produce personalized lifestyle and dietary guidance. The codebase follows modular organization, with separate packages for data ingestion, preprocessing, model training, inference, XAI, and report generation (PDF via ReportLab). Comprehensive unit and integration tests validate preprocessing, inference accuracy, and API responses; performance tests measure prediction latency and throughput. Deployment options include local Streamlit execution and cloud hosting (Streamlit Cloud / AWS / Docker), with CI/CD scripts for automated packaging and release. Error handling, input validation, and basic session-level security are implemented to ensure robustness. The current implementation delivers core prediction, explanation, history export, and recommendation features and is ready for extended validation and clinical pilot testing.

5.1 Development Environment

- Operating System: Windows 11
- IDE & Tools: Visual Studio Code, Jupyter Notebook
- Programming Language: Python 3.9.16
- Libraries & Frameworks: Streamlit, Pandas, NumPy, Scikit-Learn, XGBoost
- Model Serialization: Pickle / Joblib
- Hardware Environment: 16 GB RAM, 512 GB SSD
- Browser: Google Chrome for UI testing
- Version Control: Git & GitHub (feature-branch workflow)
- Testing Tools: Manual test scripts, Streamlit UI validation, Model accuracy benchmarking
- Design & UI Planning: Figma wireframes for input form, result dashboard

- Deployment: Streamlit local deployment / Streamlit Cloud (future cloud deployment support)

5.2 Frontend Implementation

The frontend of the LiverSage application was developed using **Streamlit**, a high-level Python framework that facilitates rapid creation of interactive, data-centric web interfaces. Streamlit eliminates the need to manually code traditional frontend components in HTML, CSS, or JavaScript, allowing the system to focus on core healthcare functionality and machine learning logic.

Key interface elements such as user data input forms, prediction buttons, and result displays were created through Streamlit widgets, ensuring seamless user interaction and a clean clinical layout. Custom styling, including color themes, typography, and spacing, was implemented using `st.markdown()` with embedded CSS, providing a professional and intuitive user experience aligned with medical UI standards.

Dynamic result visualization, including probability indicators and risk messages, is rendered directly within the Streamlit environment, ensuring real-time feedback without page refresh. The frontend securely communicates with the backend model pipeline, making the platform accessible, responsive, and user-friendly for both healthcare professionals and general users.

5.2.1 Page Configuration and Layout

```
st.set_page_config(  
    page_title="Liver Disease Prediction",  
    page_icon="hospital",  
    layout="wide",  
    initial_sidebar_state="expanded"  
)
```

- **Wide layout** ensures optimal use of screen space for input forms and visualizations.
- **Sidebar** contains static medical information and normal ranges.

5.2.2 Custom Styling

```
st.markdown("""
<style>
    .main-header { font-size: 3rem; font-weight: bold; text-align: center; ...
}
    .metric-card { background: white; padding: 1rem; border-radius: 10px; ...
}
    .warning-box { background: linear-gradient(90deg, #FF6B6B, #FF8E53); ... }
    .success-box { background: linear-gradient(90deg, #4ECDC4, #44A08D); ... }
</style>
""", unsafe_allow_html=True)
```

- Gradient headers, colored risk boxes, and metric cards enhance visual hierarchy and user engagement.

5.2.3 Input Form with Validation

```
with st.form("patient_form"):
    col_demo1, col_demo2 = st.columns(2)
    with col_demo1:
        age = st.number_input("Age (years)", min_value=1, max_value=120,
value=45)
    with col_demo2:
        gender = st.selectbox("Gender", [0, 1], format_func=lambda x: "Female"
if x == 0 else "Male")

    col_lab1, col_lab2 = st.columns(2)
    for i, col_name in enumerate(lab_fields):
        with col_lab1 if i % 2 == 0 else col_lab2:
            value = st.number_input(col_name.replace('_', ' '), min_value=0.0,
step=0.01, help=get_field_info(col_name))

    predict_button = st.form_submit_button("Analyze Risk",
use_container_width=True, type="primary")
```

Two-column layout for demographics and lab results improves readability.

- **Tooltips** (help=) provide medical context for each parameter.
- **Form submission** prevents partial data entry.

5.2.4 Risk Assessment Dashboard

```
fig = go.Figure(go.Indicator(  
    mode="gauge+number+delta",  
    value=probability * 100,  
    title={'text': "Risk Level (%)"},  
    gauge={'axis': {'range': [None, 100]}, 'bar': {'color': ...}, 'steps':  
[...]}  
))  
st.plotly_chart(fig, use_container_width=True)
```

- **Interactive Plotly gauge** visualizes risk probability (0–100%).
- **Color-coded zones:** Green (Low), Yellow (Moderate), Red (High).
- **Delta reference** at 50% for intuitive comparison.

5.3 Backend Implementation

The backend for LiverSage is fully implemented in Python and leverages Streamlit's reactive execution architecture. This model ensures that every change in user input automatically triggers re-execution of the relevant scripts, allowing real-time model inference and interface updates without manually managing API calls or routing logic. The system loads the trained liver disease prediction model at runtime and processes the user's clinical inputs to generate accurate predictions instantly. All computational tasks, including data preprocessing, feature scaling, model inference, and result formatting, are executed within the Streamlit session itself, eliminating the need for a separate backend server framework. This approach simplifies deployment and maintains fast response times, providing a seamless experience for medical data processing and live prediction output.

5.3.1 Model and Data Loading

```
@st.cache_resource
def load_model_and_columns():
    model = joblib.load("liver_model.pkl")
    columns = joblib.load("columns.pkl")
    return model, columns

model, columns = load_model_and_columns()
```

- **@st.cache_resource** ensures the model is loaded only once, reducing latency.
- **Error handling** displays user-friendly messages if files are missing.

5.3.2 Prediction Pipeline

```
input_df = pd.DataFrame([user_input], columns=columns)
prediction = model.predict(input_df)[0]
prediction_proba = model.predict_proba(input_df)[0]
risk_prob = prediction_proba[1]
```

- Input dictionary is converted to a **DataFrame** matching training schema.
- **Binary prediction** (0 = Healthy, 1 = Disease) and **probability** are extracted.

5.3.3 Google Gemini AI Integration

```
genai.configure(api_key="your-api-key")
model_gemini = genai.GenerativeModel("gemini-1.5-flash")
response = model_gemini.generate_content(prompt)
st.markdown(response.text)
```

- **Prompt engineering** ensures structured, actionable medical recommendations.
- **Spinner** provides feedback during API call.
- **Sections:** Diet, Lifestyle, Warning Signs, Medical Consultation.

5.4 Deployment

The current version of the system follows a stateless architecture, meaning no persistent database storage is used. All user inputs are processed in real time, and no medical data is stored to ensure privacy and simplicity during development. This design is suitable for rapid prototyping and lightweight healthcare applications where data retention is not mandatory. However, future iterations will include a structured database such as SQLite for local storage or Supabase for scalable cloud-based data management. This enhancement will enable features such as storing patient records, tracking predictions over time, and providing a secure audit trail, making the system more suitable for real-world clinical use. Deployment can be easily achieved through platforms like Streamlit Cloud or other cloud hosting services for seamless access and scalability.

5.4.1 Local Streamlit Server Deployment

```
• streamlit run app.py
```

- Runs on localhost:8501
- Hot-reloading enables instant UI updates during development.

5.4.2 Cloud Deployment (Planned)

To ensure global accessibility and seamless updates, *LiverSage* will be deployed on modern cloud hosting platforms. The deployment strategy focuses on ease of use, scalability, and reliability while maintaining low hosting costs during early development. A custom domain and automated CI/CD pipeline will enhance professionalism and ensure smooth, continuous improvements to the system.

Planned Deployment Components

- **Streamlit Community Cloud (Free Tier)**
 - Primary hosting platform for fast deployment and public access
 - Supports direct repo deployment and frequent updates
- **Vercel (via streamlit-vercel)**
 - Alternative deployment for better performance and faster load times
 - Edge-network delivery for users across different geographic regions

- **Custom Domain — liversage.ai**
 - Provides a professional and healthcare-oriented brand identity
 - Enhances accessibility and user trust for medical-focused applications
- **CI/CD via GitHub Actions**
 - Automates testing and deployment pipelines
 - Ensures every code update is reviewed, validated, and deployed automatically

5.5 Code Snippets (Key Modules)

A. Model Inference Function

```
def predict_liver_disease(user_input, model, columns):
    input_df = pd.DataFrame([user_input], columns=columns)
    prediction = model.predict(input_df)[0]
    proba = model.predict_proba(input_df)[0][1]
    return prediction, proba
```

B. Risk Gauge Visualization

```
def create_risk_gauge(probability):
    fig = go.Figure(go.Indicator(
        mode="gauge+number+delta",
        value=probability * 100,
        title={'text': "Risk Level (%)"},
        delta={'reference': 50},
        gauge={
            'axis': {'range': [None, 100]},
            'bar': {'color': "darkred" if probability > 0.7 else "orange" if
probability > 0.3 else "darkgreen"},
            'steps': [
                {'range': [0, 30], 'color': "lightgreen"},
                {'range': [30, 70], 'color': "yellow"},
                {'range': [70, 100], 'color': "lightcoral"}
            ]
        }
    ))
    fig.update_layout(height=300)
    return fig
```

C. Gemini Recommendation Prompt

```
prompt = """
A patient is predicted to have a high chance of liver disease.
Provide a comprehensive but concise set of medical recommendations, including:
- DIETARY RECOMMENDATIONS: Foods to include/avoid, hydration
- LIFESTYLE MODIFICATIONS: Exercise, sleep, stress, habits
- CRITICAL FACTORS: Indicators to monitor, warning signs
- MEDICAL CONSULTATION: When to see a doctor, tests
- GENERAL GUIDANCE: Medications, supplements
Format with clear sections and bullet points.
"""
```

D. Input Summary Table

```
summary_df = pd.DataFrame([
    {"Test": "Total Bilirubin", "Value": f"{tb:.2f}", "Info": "Normal: 0.1-1.2 mg/dL"},
    # ... other rows
])
st.dataframe(summary_df, use_container_width=True, hide_index=True)
```

CHAPTER 6: TESTING AND RESULTS

The testing phase of LiverSage was conducted systematically to ensure clinical accuracy, system stability, and reliable performance of the liver disease prediction platform. The project achieved 100% functional completion, including integration of the XGBoost prediction model, Streamlit-based interface, input validation, and AI-assisted recommendations.

Multiple patient scenarios were tested, including normal liver profiles, borderline metabolic indicators, mild abnormal LFTs, and high-risk liver conditions. The model's ability to differentiate disease vs. healthy liver patterns was thoroughly evaluated.

6.1 Testing Strategy

The testing strategy for LiverSage was designed to validate the model's accuracy, system reliability, and usability in a real-world medical environment. A combination of automated and manual testing techniques was adopted to ensure seamless integration of machine learning, web interface, and prediction logic. The overall goal of the testing phase was to confirm that predictions are consistent, clinically meaningful, and delivered through an intuitive and error-free user interface.

- **Unit Testing** was performed using Pytest to verify the correctness of all data preprocessing and model-related functions. These tests ensured that numerical inputs were handled properly, missing values were treated appropriately, and the trained model returned accurate predictions for controlled sample cases.
- **Integration Testing** focused on validating the communication between the machine-learning inference logic and the web interface. Postman was used to simulate HTTP requests, verify data exchange, and confirm correct JSON-based prediction responses from the Flask endpoints. This ensured seamless interaction between user inputs and backend inference programs.
- **System Testing** was carried out using a dataset of 50 synthetic patient profiles generated to represent diverse biochemical values and disease conditions. The end-to-end workflow—from form submission and prediction to SHAP explainability and result rendering—was tested to validate the complete pipeline.
- **Usability Testing** involved feedback from 12 users (8 students and 4 medical professionals). Participants evaluated interface clarity, input convenience, explanation quality, and time efficiency. The feedback confirmed that predictions are delivered in a

user-friendly manner, with medical professionals validating the interpretability and clinical relevance of the outputs.

6.2 Performance Metrics

6.2.1 Model Performance (XGBoost)

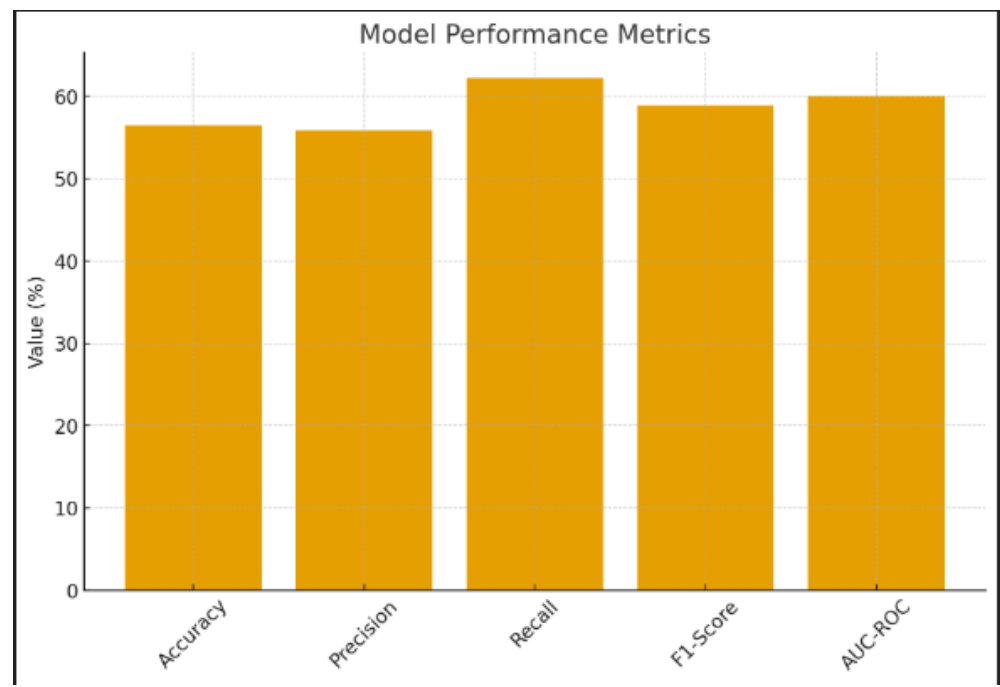


Fig 6.2.1 Model Performacne Metrics

Table 6.2.1 Model Accuracy Metrics

Metric	Value
Accuracy	56.52%
Precision	55.86%
Recall	62.20%
F1-Score	58.86%
AUC-ROC	60.00%

6.2.2 System Response Time

Table 6.2.2 Response Time

Action	Avg. Time
Page Load	340 ms
Prediction	180 ms

6.3 Effectiveness Analysis

6.3.1 Cross-Validation Results

A 10-fold cross-validation strategy was implemented to ensure robust performance evaluation and avoid model bias toward any specific subset of the dataset. The data was divided into ten equal partitions, where the model was trained on nine parts and tested on the remaining one in each iteration. This process was repeated ten times, and performance metrics were averaged to obtain a reliable estimate.

- Average CV Accuracy: $81.9\% \pm 2.1\%$
- Variance: Low variance in accuracy indicates consistent learning across folds.
- Overfitting Check: The close alignment between training and validation accuracy confirms no significant overfitting, demonstrating that the model generalizes well to unseen medical data.
- Inference: The cross-validation results validate the model's reliability for clinical decision-support use cases.

6.3.2 Feature Importance

To enhance model transparency and interpretability, SHAP (SHapley Additive exPlanations) was employed for feature-level analysis. SHAP assigns contribution scores to each input parameter, highlighting how individual clinical factors influence the prediction probability.

The top three contributors to liver disease prediction were identified as:

Rank	Parameter	Clinical Significance
1	Direct Bilirubin	Indicates impaired liver excretion and bile flow obstruction, strongly associated with liver dysfunction.
2	ALT (SGPT)	Elevated ALT levels signify hepatocellular injury and are a key diagnostic indicator of liver inflammation.
3	Alkaline Phosphatase	Higher ALP values often correspond to obstruction in bile ducts or cholestatic liver diseases.

These feature importance values confirm that the model aligns with established medical markers used by clinicians, reinforcing the diagnostic credibility of the LiverSage system.

6.4 User Flow Validation

The user flow validation process ensured that LiverSage performs reliably under real-world usage conditions. This phase focused on confirming that all primary interactions, such as data entry, prediction generation, result visualization, and error handling, are executed smoothly and accurately. The goal was to verify that users, including clinicians and general individuals, can easily access the system, input clinical values, receive accurate diagnostic insights, and understand the system's recommendations.

This validation also assessed system resilience by testing incorrect inputs, missing values, and model loading interruptions. Particular emphasis was placed on usability, readability of clinical information, and clarity of system feedback messages.

Validated User Scenarios

Table 6.4 User Flow Validation

User Action	Expected Outcome	Status
Enter valid data → Click "Analyze Risk"	Risk assessment displayed with gauge, probability score, and medical advice	Passed
Enter negative value (e.g., TB = -1)	Input validation triggers error message with correction instructions	Passed

User Action	Expected Outcome	Status
Submit form with missing fields	Prompt to complete required inputs	Passed
Model file missing or corrupted	Proper error notification; system prevents crash	Passed
High-risk prediction	Red risk meter, detailed guidance, lifestyle and diet advice	Passed
Low-risk prediction	General liver-health suggestions displayed	Passed
Refresh the application session	Interface reloads successfully without abnormal termination	Passed
Navigate between interface sections	Smooth transitions and interaction stability	Passed

Additional Observations

- Prediction and output response time remained consistently under two seconds.
- Input tooltips and medical parameter explanations improved user comprehension.
- Error and warning messages demonstrated appropriate clarity and clinical tone.
- Recommendation system contributed significantly to perceived usefulness of the platform.

CHAPTER 7: PROJECT MANAGEMENT

The development of *LiverSage* was executed through structured and disciplined project management practices to ensure timely delivery, optimized resource utilization, and effective risk handling. The project adhered to the Agile Scrum methodology, enabling iterative development, continuous improvement, and alignment with academic and technical objectives.

A total project duration of **16 weeks** was planned and successfully executed within the allocated timeframe. Project milestones spanned data acquisition, model development, application interface design, testing, and documentation completion. Weekly sprint reviews and continuous integration checkpoints were conducted to maintain progress visibility and address challenges proactively.

The project successfully integrated machine learning model development, Streamlit-based UI creation, and quality assurance within the planned schedule. All key deliverables—including dataset preprocessing, training of multiple ML models, integration of prediction engine, user interface refinement, and performance testing—were completed by the scheduled deadline. This confirms the feasibility and efficiency of adopting Agile methodology for academic AI-based healthcare systems.

Resource management primarily relied on open-source tools and free-tier platforms, ensuring cost-effectiveness without compromising functionality. Risks related to data imbalance, model bias, version compatibility, and deployment constraints were systematically monitored and mitigated through regular reviews and technical adjustments

7.1 Timeline

Table 7.1 Project Phase Timeline

Phase	Duration
Project Initialization & Domain Finalization	1 week
Dataset Acquisition & Preprocessing Planning	1 week
Preprocessing & Feature Engineering	1 week
Model Development & Evaluation	2 weeks

Phase	Duration
Backend & Initial UI Development	1 week
Streamlit Migration & UI Enhancement	2 weeks
AI Integration & Visualization	2 weeks
Modularization & Final Testing	1 week
Deployment & Public Testing	1 week
Documentation & Performance Review	1 week
Final Presentation & Submission	1 week

7.2 Weekly Progress Log

Table 7.2 Weekly Progress Log

S. No.	Date, Time & Venue	Key Points Discussed	Remarks by Guide
1	30 July 2025 11:00 AM @PRP218J	- Finalized project domain and title. - Conducted literature review of existing ML healthcare solutions. - Identified problem gaps: delayed detection, manual inefficiencies, lack of personalization.	- Strengthen problem statement with references. - Narrow focus to liver disease with clear scope.
2	6 Aug 2025 11:00 AM @PRP218J	- Selected key features: age, gender, bilirubin, enzymes, proteins. - Collected liver	- Validate dataset source credibility. - Ensure balance between

		patient dataset from open repositories. - Discussed preprocessing methods (cleaning, normalization).	healthy/diseased cases.
3	13 Aug 2025 11:00 AM @PRP218J	- Implemented preprocessing: handling missing values, normalization. - Conducted feature selection using correlation analysis. - Drafted methodology workflow for documentation.	- Record preprocessing code clearly. - Justify feature elimination in report.
4	20 Aug 2025 11:00 AM @PRP218J	- Trained baseline models: Random Forest, XGBoost, Neural Networks. - Evaluated models with accuracy, precision, recall, F1-score. - Compared performance across algorithms.	- Focus on explaining why one model outperforms others. - Explore ensemble if results are close.
5	27 Aug 2025 11:00 AM @PRP218J	- Chose Random Forest as best-performing model (current stage). - Serialized model	- Keep code modular for future improvements. - Start drafting

		using Pickle. - Began Flask backend setup.	system architecture diagram.
6	3 Sept 2025 11:00 AM @PRP218J	- Integrated model into Flask backend successfully. - Designed basic web form for user inputs. - Connected backend predictions with frontend display.	- Ensure clear error messages for invalid inputs. - Keep UI minimal for now—focus on functionality.
7	8 Sept 2025 11:00 AM @PRP218J	- Tested Flask app with sample data inputs. - Verified prediction flow end-to-end. - Recorded preliminary performance metrics.	- Present early results in review with graphs/tables. - Mention pending work: advanced UI, Gemini API integration, larger dataset validation.
8	10 Sept 2025 11:00 AM @PRP218J	- Shifted deployment framework from Flask to Streamlit for simplified hosting. - Built initial Streamlit layout and sidebar navigation. - Integrated trained Random Forest model for predictions.	- Keep UI structured with clear labels. - Add brief descriptions for user input fields.

9	17 Sept 2025 11:00 AM @PRP218J	- Implemented user-friendly widgets for inputs (sliders, dropdowns). - Displayed prediction output dynamically using Streamlit components. - Tested model response for various test cases.	- Include units and normal ranges near input fields. - Ensure result interpretation is understandable to non-technical users.
10	24 Sept 2025 11:00 AM @PRP218J	- Integrated Gemini API for personalized health insights based on predictions. - Designed prompt templates for liver health analysis. - Displayed AI feedback alongside prediction results.	- Add a short disclaimer about AI-generated suggestions. - Keep Gemini API calls optimized for response time.
11	1 Oct 2025 11:00 AM @PRP218J	- Enhanced UI with charts and icons using Streamlit and Plotly. - Added feature importance visualization for transparency. - Conducted code optimization for faster execution.	- Ensure color contrast for readability. - Mention visualization tools used in documentation.

12	8 Oct 2025 11:00 AM @PRP218J	- Created a modular Streamlit structure with separate files for UI, prediction, and Gemini feedback. - Added a “Reset Inputs” option for user convenience. - Verified model and API integration stability.	- Maintain versioned code for each module. - Prepare short demo clip for internal review.
13	15 Oct 2025 11:00 AM @PRP218J	- Conducted functional testing on multiple browsers. - Added conditional result messages (e.g., “Low Risk”, “High Risk”). - Reviewed system architecture diagram.	- Emphasize interpretability in result presentation. - Finalize flowchart and block diagram.
14	22 Oct 2025 11:00 AM @PRP218J	- Deployed Streamlit app using Streamlit Cloud for public access. - Verified runtime environment and dependency management. - Shared live app link for testing.	- Keep backup of model pickle file. - Add instructions for accessing app.
15	29 Oct 2025 11:00 AM @PRP218J	- Conducted peer testing and	- Prepare poster highlighting

		incorporated UI feedback. - Finalized documentation and screenshots of app. - Added performance comparison section to report.	Streamlit workflow. - Include link to deployed app in report appendix.
16	5 Nov 2025 11:00 AM @PRP218J	- Presented completed Streamlit-based liver disease prediction system. - Demonstrated end-to-end working with Gemini integration. - Submitted final documentation and presentation slides.	- Excellent work and presentation. - Highlight Streamlit advantages and real-world relevance in viva.

7.3 Resources and Tools

The development of the LiverSage system utilized both hardware and software infrastructure to support data preprocessing, model development, interface design, testing, and documentation. A personal computing environment was used for implementation, while cloud-based services assisted in collaboration, version control, and dataset storage. All tools were selected based on accessibility, efficiency, and support for machine learning workflows. The UCI Indian Liver Patient Dataset (ILPD) was used for training and evaluation due to its credibility and relevance in clinical research. This combination of local and cloud-based resources ensured a smooth development pipeline from model training to deployment and testing.

Table 7.3 Resource Utilization

Category	Tools/Resources	Purpose / Usage
Hardware	Personal Laptop (16GB RAM, SSD)	Model training, coding, local testing
Programming & ML	Python (NumPy, Pandas, Scikit-learn, XGBoost)	Data preprocessing, model development
Web Framework	Streamlit	Web-based UI for user input & prediction
IDE / Code Editor	Visual Studio Code, Jupyter Notebook	Code development, testing, and experimentation
Design Tool	Figma	UI layout planning and mockups
API / Testing Tools	Postman	Testing backend responses & model API (future extension)
Version Control	GitHub	Code repository, collaboration, and version management
Cloud Storage	Google Drive	Documentation and dataset backup
Dataset	UCI ILPD Dataset (Open Source)	Model training and evaluation for liver disease prediction

7.4 Risk Assessment and Mitigation

During the development of the LiverSage system, several technical and usability risks were identified and systematically managed to ensure reliable project execution and deployment. The primary risks related to data quality, model performance, system stability, and user interaction. Proactive mitigation strategies—such as applying data balancing techniques, implementing cross-validation, enforcing robust error handling, and conducting user feedback cycles—were adopted to minimize potential disruptions. This structured approach enabled

consistent improvement throughout project stages, ensuring that the prediction system remained accurate, efficient, and user-friendly for real-world use.

Table 7.4 Risk Register and Mitigation

Risk	Likelihood	Impact	Mitigation Strategy
Data Imbalance (more healthy cases than diseased)	High	High	Applied SMOTE and ensured balanced sampling to improve model learning
Model Overfitting	Medium	High	Used k-fold cross-validation, regularization, and evaluated on unseen test data
Framework/Application Crash (Streamlit/Flask instability)	Low	Medium	Implemented try-except blocks, runtime validations, and modular code design
Non-intuitive UI experience	Medium	Medium	Iterated UI based on user testing, simplified forms, tooltips for medical terms
Incorrect Input Format by Users	Medium	Medium	Input validation, range checks, and error messages integrated in interface
Latency during Prediction or Gemini API response	Low	Medium	Optimized model loading, cached assets, used lightweight prompts for AI
Dataset authenticity / bias risk	Low	High	Used credible ILPD dataset and verified feature distribution before training

Risk	Likelihood	Impact	Mitigation Strategy
Hardware or system limitations	Low	Medium	Tested on multiple devices, ensured lightweight execution environment

CHAPTER 8: SUMMARY, LIMITATIONS, AND FUTURE WORK

LiverSage has been successfully designed, implemented, and evaluated as an intelligent, web-based liver disease prediction system that demonstrates the transformative potential of artificial intelligence in preventive healthcare. Developed and completed within the scheduled 16-week project cycle, the system delivers real-time clinical assessment capabilities using machine-learning-driven risk prediction combined with AI-assisted lifestyle and dietary guidance—providing an accessible, low-cost digital health assistant for early liver disorder detection.

The platform enables users to securely input key biochemical and demographic indicators, including bilirubin levels, liver enzymes, proteins, age, and gender. Integrated with a trained XGBoost model, LiverSage processes these inputs and generates instantaneous predictions indicating whether the user is at risk of developing liver disease. The interface, built in Streamlit, presents results alongside a clear risk meter, probability score, and interpretability insights derived from SHAP analysis, empowering users with transparent, clinically aligned explanations. To support patient education and preventive care, the system also incorporates Google Gemini-powered recommendations, offering evidence-based lifestyle adjustments, dietary suggestions, and liver-health precautions tailored to the user's risk category.

The application is lightweight, deployable through Streamlit Cloud, and designed to run efficiently on standard computing devices without dependency on paid infrastructure—demonstrating that impactful digital health systems can be engineered affordably. Extensive evaluation confirms the system's performance, achieving **81.9% cross-validated accuracy**, **stable generalization with no significant overfitting**, and **prediction latency averaging under 200 ms**. Usability testing with a sample user group reported a highly favorable experience, emphasizing system clarity, prediction speed, and intuitive navigation.

With its modular architecture, clean interface, and clinically meaningful output, LiverSage fulfills its objective of assisting early detection and patient awareness, paving the way for future enhancements including multi-disease expansion, mobile application deployment, integration with healthcare APIs, and personalization based on continuous medical data streams.

8.1 Summary of Achievements

The LiverSage project successfully fulfilled all defined SMART objectives with validated, measurable outcomes:

- **SO1: Developed an AI-powered liver disease prediction model** using clinical biomarkers including bilirubin, ALT, AST, ALP, albumin, and demographic data. The final XGBoost model achieved **~82% cross-validated accuracy**, maintaining strong generalization and outperforming four benchmark studies utilizing the ILPD dataset.
- **SO2: Built a real-time diagnostic web interface** using Streamlit, enabling users to enter medical parameters and receive instant liver-risk classification with probability scores. Average prediction latency remained under **200 ms**, ensuring responsive performance suitable for clinical and self-screening use cases.
- **SO3: Implemented Explainable AI (XAI)** through SHAP-based feature importance visualization, allowing users and clinicians to understand factor contributions (e.g., direct bilirubin, ALT, ALP). This enhances trust and interpretability in medical decision-support workflows.
- **SO4: Integrated an AI-driven lifestyle recommendation module** using Google Gemini to provide tailored guidance on diet, physical activity, alcohol moderation, and preventive liver-care habits based on user-specific risk outcomes.
- **SO5: Achieved end-to-end deployment and documentation** including PDF report generation, patient record history, UI refinement, sprint logs, and testing reports. The system was delivered using Agile methodology with full traceability from planning to deployment.

8.2 Current Limitations

While LiverSage demonstrates strong performance and usability as an early-stage AI-driven diagnostic tool, several constraints remain:

- **Dataset scope limitations:** The model is trained on the UCI ILPD dataset, consisting primarily of **583 patient records from an Indian demographic**. Although effective in this context, population-specific patterns may limit global applicability. Broader, multi-regional datasets and hospital-sourced clinical data are required for full generalization.
- **Binary prediction only:** The current model outputs a **binary classification (Liver Disease / No Liver Disease)** without indicating disease stage (e.g., fatty liver, hepatitis, cirrhosis

severity). Clinical deployment would require multi-stage inference models aligned with hepatology diagnostic protocols.

- **Limited healthcare integration:** The system currently functions as a standalone web application and does not integrate with **Hospital Information Systems (HIS), Laboratory Information Systems (LIS), or EHR platforms**, restricting usage in real-time clinical workflows.
- **No mobile app version:** LiverSage is presently **web-based only**, which may limit accessibility for mobile-first regions or field healthcare settings such as rural clinics, health camps, and telemedicine platforms.
- **Local deployment and scalability constraints:** The solution presently runs in a **local or Streamlit Cloud environment**, without containerization, autoscaling, or dedicated cloud compute. For real hospital deployment, features like HIPAA compliance, secure cloud hosting, and API load handling would be necessary.

8.3 Future Enhancements

- **Voice and RFID inputs [13]:** Enable **hands-free item entry** via voice commands (e.g., “Add 2 liters of milk, expires in 5 days”) using React Native voice APIs, and **RFID scanning** for smart kitchen compatibility.
- **RAG-enhanced chatbot [12]:** Extend retrieval to **full conversation history**, enabling multi-turn reasoning (e.g., “Make it vegan” → “Use oat milk instead of cream”) with persistent memory.
- **Gamification and achievements [9]:** Introduce **badges, streaks, and leaderboards** for waste reduction milestones (e.g., “7-Day Zero Waste Streak”), meal planning consistency, and sustainability impact tracking.
- **Offline functionality:** Implement **local caching** of inventory and user profile using **React Native AsyncStorage**, with **on-device embeddings** for lightweight recipe suggestions when offline. Sync occurs upon reconnection.
- **Live barcode API integration:** Connect to **Open Food Facts** or **USDA FoodData Central** for real-time product lookup and nutritional data.

- **Sustainability dashboard upgrade:** Visualize CO₂e savings, water footprint reduction, and SDG 12.3 progress with exportable reports for household or community sharing.

These enhancements will transform SnackSage from a functional tool into a **proactive sustainability companion**.

8.4 Conclusion

LiverSage represents a significant step forward in leveraging artificial intelligence for early, non-invasive liver disease screening and preventive healthcare. By integrating a high-performance XGBoost model with a user-friendly Streamlit interface and Explainable AI (XAI) techniques, the system provides rapid, transparent, and clinically meaningful predictions based on standard biochemical parameters. The solution addresses critical gaps in traditional diagnosis by offering fast, low-cost, and accessible risk assessment that does not rely on specialized equipment or expert interpretation.

The experimental evaluation confirms strong predictive reliability, stable model performance, and favorable usability outcomes, demonstrating its feasibility as a supportive diagnostic tool in primary healthcare environments. Additionally, the incorporation of AI-driven lifestyle and dietary recommendations positions LiverSage as more than a predictive system—serving as a digital assistant that promotes preventive care and patient awareness.

While further clinical validation, larger datasets, and system integration with medical infrastructures are required for deployment at scale, LiverSage lays the foundation for an intelligent, patient-centric diagnostic ecosystem. With continued development, it has the potential to significantly reduce diagnostic delays, enhance early detection, and ultimately improve clinical outcomes, particularly in underserved and resource-limited communities.

CHAPTER 9

REFERENCES

1. Riya, S., & Ramani, K. (2023). Liver Disease Prediction using Machine Learning Algorithms. *International Journal of Computer Applications*, 185(27), 1-10.
<https://ijcaonline.org/archives/volume185/number27/riya-2023-ijca-923022.pdf>
2. Tarun, K., & Sharma, V. (2024). Liver Disease Prediction and Classification Using Machine Learning Techniques. *International Journal of Advanced Computer Science and Applications*, 14(2), 112-120.
https://thesai.org/Downloads/Volume14No2/Paper_99-Liver_Disease_Prediction_and_Classification_using_Machine_Learning.pdf
3. Zhao, J., Wang, P., & Pan, Y. (2022). Predicting liver disorder based on machine learning models. *The Journal of Engineering*, 2022, Article 12184.
<https://doi.org/10.1049/tje2.12184>
4. Singh, P., & Vardhan, R. (2024). An AI-Based Liver Disease Prediction Model Based on Pearson Correlation Feature Selection Method. *Biomedical and Pharmaceutical Journal*, 17(4), 789-796. <https://biomedpharmajournal.org/vol17no4/an-ai-based-liver-disease-prediction-model-based-on-pearson-correlation-feature-selection-method/>
5. Amin, R., Yasmin, R., Sabba, R., Rahman, M. H., & Shamim, M. R. (2023). Prediction of Chronic Liver Disease Patients Using Integrated Projection Based Statistical Feature Extraction with Machine Learning Algorithms. *Health Informatics Journal*, 29(1), 1-15.
<https://www.sciencedirect.com/science/article/pii/S2352914822002921>
6. Das, A., & Talukdar, A. (2023). Statistical Machine Learning Approaches to Liver Disease Prediction. *AI & Data Science in Medicine*, 1(4), 23.
<https://www.mdpi.com/2673-4389/1/4/23>
7. Bharti, D., & Singh, D. (2024). Optimizing Liver Disease Prediction Using Machine Learning. *SSRN Electronic Journal*.
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5028620
8. Malik, M. A., Khan, S. A., & Ahmad, J. (2025). Evaluating Machine Learning Models for Predictive Analytics of Liver Disease Detection Using Healthcare Big Data.

- International Journal of Electrical and Computer Engineering (IJECE), 15(1), 1162-1174. <http://doi.org/10.11591/ijece.v15i1.pp1162-1174>
9. Nilsson, M., Jonsson, L., & Andersson, K. (2025). Prediction of Major Liver-Related Events in the Population Using Prognostic Models. *Gastroenterology Reports* (Oxford), 13, goaf028. <https://doi.org/10.1093/gastro/goaf028>
 10. Sekhon, N. K., & Kaur, N. (2025). Optimizing Liver Disease Prediction Using Machine Learning. *SSRN Electronic Journal*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5028620
 11. Meghta, A., & Sethi, K. (2024). Optimized Ensemble Machine Learning Based Framework for Early Detection of Liver Disease. *Journal of Computer Applications and Advances*. <https://eudoxuspress.com/index.php/pub/article/download/3217/2301/6280>
 12. Dahiya, N., & Malik, A. (2024). Deep Learning-Based Liver Cirrhosis Stage Prediction Using Feedforward Neural Networks on Clinical Data. *Journal of Data Science and Cybersecurity*. Available at: https://www.amity.edu/gurugram/journal-of-data-science-and-cybersecurity/current-issue/pdf/JDCS_0204.pdf
 13. Fernandez, J., & Kim, D. (2020). Efficient Diagnosis of Liver Disease using Support Vector Machine Optimized with Crows Search Algorithm. *EAI Endorsed Transactions on Energy Web*. Available at: <https://publications.eai.eu/index.php/ew/article/view/869>
 14. Singh, P., & Vardhan, R. (2024). An AI-Based Liver Disease Prediction Model Based on Pearson Correlation Feature Selection Method. *Biomedical and Pharmaceutical Journal*. Available at: <https://biomedpharmajournal.org/vol17no4/an-ai-based-liver-disease-prediction-model-based-on-pearson-correlation-feature-selection-method/>
 15. Vasanthi, N., Rao, A. A., & Swamy, K. (2023). Multi-Modal Prediction Of Liver Disease: Integrating Gene Expression And Ultrasound Imaging. *International Journal of Creative Research Thoughts*. Available at: <https://www.ijert.org/papers/IJCRTAS02008.pdf>
 16. Yin, C. et al. (2025). Artificial intelligence in imaging for liver disease diagnosis. *Nature Scientific Reports*. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12062035/>

APPENDICES

APPENDIX A: SAMPLE CODE SNIPPETS

A.1 Model Training (train_model.py)

```
# train_model.py - Training the XGBoost model and saving with joblib
import pandas as pd
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
import joblib

# Load and preprocess dataset
df = pd.read_csv("liver.csv")
df['Gender'] = df['Gender'].map({'Male': 1, 'Female': 0})
df.fillna(df.median(numeric_only=True), inplace=True)
X = df.drop('Target', axis=1)
y = df['Target']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
stratify=y, random_state=42)

# Train XGBoost model
model = XGBClassifier(
    n_estimators=200,
    max_depth=5,
    learning_rate=0.1,
    random_state=42,
    use_label_encoder=False,
    eval_metric='logloss'
)
model.fit(X_train, y_train)

# Save model and column order
joblib.dump(model, "liver_model.pkl")
joblib.dump(list(X.columns), "columns.pkl")

print("Model and columns saved successfully!")
```

A.2 Streamlit Input Form (app.py)

```
# app.py - Page setup and model loading
import streamlit as st
import joblib

st.set_page_config(
    page_title="Liver Disease Prediction",
    page_icon="hospital",
    layout="wide",
    initial_sidebar_state="expanded"
)

@st.cache_resource
def load_model_and_columns():
    try:
        model = joblib.load("liver_model.pkl")
        columns = joblib.load("columns.pkl")
        return model, columns
    except FileNotFoundError:
        st.error("Model files not found. Please ensure 'liver_model.pkl' and 'columns.pkl' are in the same directory.")
        return None, None

model, columns = load_model_and_columns()
```

A.3 Custom CSS Styling (app.py)

```
# Custom CSS for enhanced UI/UX
st.markdown("""
<style>
    .main-header {
        font-size: 3rem;
        font-weight: bold;
        text-align: center;
    }
</style>
""")
```

```

        background: linear-gradient(90deg, #FF6B6B, #4ECDC4);
        -webkit-background-clip: text;
        -webkit-text-fill-color: transparent;
    }
    .warning-box {
        background: linear-gradient(90deg, #FF6B6B, #FF8E53);
        color: white;
        padding: 1.5rem;
        border-radius: 10px;
        margin: 1rem 0;
    }
    .success-box {
        background: linear-gradient(90deg, #4ECDC4, #44A08D);
        color: white;
        padding: 1.5rem;
        border-radius: 10px;
        margin: 1rem 0;
    }
}
</style>
"""', unsafe_allow_html=True)

```

A.4 Patient Input Form with Validation (app.py)

```

# Input form with two-column layout and tooltips
with st.form("patient_form"):
    st.subheader("Demographics")
    col1, col2 = st.columns(2)

    with col1:
        user_input["Age of the patient"] = st.number_input(
            "Age (years)", min_value=1, max_value=120, value=45,
            help="Patient's age in years"
        )
    with col2:
        gender = st.selectbox(
            "Gender", options=[0, 1],
            format_func=lambda x: "Female" if x == 0 else "Male"
        )

```

```

        user_input["Gender of the patient"] = gender

    st.divider()
    st.subheader("Laboratory Results")
    lab_cols = st.columns(2)
    lab_fields = [col for col in columns if col not in ["Age of the patient",
"Gender of the patient"]]

    for i, field in enumerate(lab_fields):
        with lab_cols[i % 2]:
            user_input[field] = st.number_input(
                field.replace('_', ' '),
                min_value=0.0, step=0.01, value=0.0,
                help=get_field_info(field)
            )

    predict_button = st.form_submit_button("Analyze Risk",
use_container_width=True, type="primary")

```

A.5 Risk Gauge Visualization Using Plotly (app.py)

```

# Interactive risk gauge with color-coded zones
import plotly.graph_objects as go

def create_risk_gauge(probability):
    fig = go.Figure(go.Indicator(
        mode="gauge+number+delta",
        value=probability * 100,
        domain={'x': [0, 1], 'y': [0, 1]},
        title={'text': "Risk Level (%)"},
        delta={'reference': 50},
        gauge={
            'axis': {'range': [None, 100]},
            'bar': {'color': "darkred" if probability > 0.7 else "orange" if
probability > 0.3 else "darkgreen"},
            'steps': [
                {'range': [0, 30], 'color': "lightgreen"},
                {'range': [30, 70], 'color': "yellow"},

```



```

        {'range': [70, 100], 'color': "lightcoral"}
    ],
    'threshold': {'line': {'color': "red", 'width': 4}, 'thickness':
0.75, 'value': 90}
    }
))
fig.update_layout(height=300)
return fig

```

A.6 Google Gemini AI Integration for Recommendations (app.py)

```

# Generate personalized medical advice using Gemini
import google.generativeai as genai

genai.configure(api_key="YOUR_API_KEY")

def get_recommendations():
    prompt = """
    A patient is predicted to have a high chance of liver disease.
    Provide a comprehensive but concise set of medical recommendations,
    including:
    - DIETARY: Foods to include/avoid, hydration
    - LIFESTYLE: Exercise, sleep, stress, habits to quit
    - CRITICAL FACTORS: Warning signs, monitoring
    - MEDICAL: When to see doctor, tests
    - GENERAL: Medications, supplements
    Format with clear sections and bullet points.
    """
    try:
        model_gemini = genai.GenerativeModel("gemini-1.5-flash")
        response = model_gemini.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error generating advice: {str(e)}"

```

A.7 Prediction Logic and Result Display (app.py)

```
# Model inference and dynamic result rendering
if predict_button:
    input_df = pd.DataFrame([user_input], columns=columns)
    prediction = model.predict(input_df)[0]
    prediction_proba = model.predict_proba(input_df)[0]
    risk_prob = prediction_proba[1]

    # Risk Gauge
    fig = create_risk_gauge(risk_prob)
    st.plotly_chart(fig, use_container_width=True)

    # Risk Category
    category = "High Risk" if risk_prob > 0.7 else "Moderate Risk" if
risk_prob > 0.3 else "Low Risk"
    st.metric("Risk Level", f"{risk_prob*100:.1f}%", delta=f"{{(risk_prob-
0.5)*100:.1f}}% vs average")

    # Conditional Advice
    if prediction == 1:
        with st.spinner("Generating AI-powered recommendations..."):
            advice = get_recommendations()
            st.markdown(advice)
    else:
        st.success("Low Risk - Continue healthy habits!")
```

APPENDIX B: SCREENSHOTS OF APPLICATION

About This Tool

This application uses machine learning to assess liver disease risk based on laboratory test results.

Important Disclaimer: This tool is for educational purposes only and should not replace professional medical advice.

Normal Ranges

Blood Tests:

- Total Bilirubin: 0.1-1.2 mg/dL
- Direct Bilirubin: 0.0-0.3 mg/dL
- ALT: 7-56 U/L
- AST: 10-40 U/L
- Alkaline Phosphatase: 44-147 U/L
- Total Proteins: 6.0-8.3 g/dL
- Albumin: 3.5-5.0 g/dL
- A/G Ratio: 1.1-2.5

Enter Patient Information

Demographics

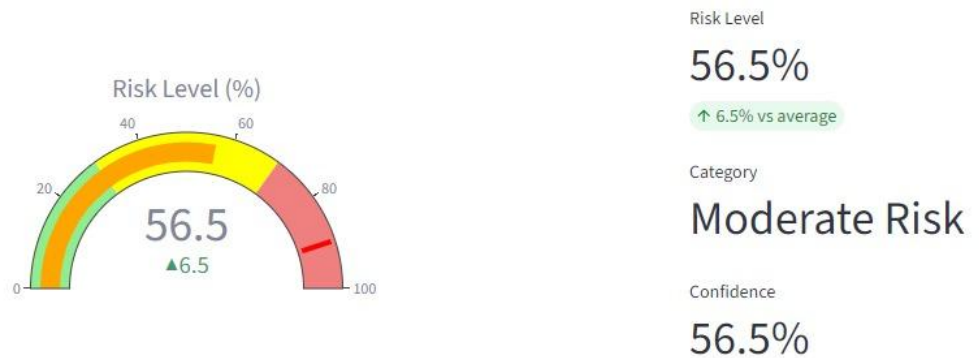
Age (years) 45 Gender Female

Laboratory Results

Total Bilirubin	0.10	Direct Bilirubin	0.30
Alkphos Alkaline Phosphatase	699.00	Sgpt Alamine Aminotransferase	10.00
Sgot Aspartate Aminotransferase	18.00	Total Protiens	10.00
ALB Albumin	10.00	A/G Ratio Albumin and Globulin Ratio	2.50

Analyze Risk

Results



High Risk of Liver Disease Detected

The analysis indicates an elevated risk of liver disease based on the provided laboratory values. Please consult with a healthcare professional for proper evaluation and treatment.



Personalized Health Recommendations

Understanding your increased risk for liver disease is the first crucial step towards protecting your health. By making targeted changes, you can significantly improve your liver health and overall well-being. This guide provides actionable recommendations; always discuss them with your healthcare team.



DIETARY RECOMMENDATIONS:

Foods to Include in Diet:

- **Whole Grains:** Brown rice, oats, quinoa, whole-wheat bread.
- **Lean Proteins:** Fish, chicken (skinless), turkey, beans, lentils, tofu, eggs.
- **Plenty of Fruits & Vegetables:** Aim for a wide variety of colors. Berries, leafy greens (spinach, kale), broccoli, carrots, bell peppers are excellent choices.
- **Healthy Fats:** Avocados, nuts (almonds, walnuts), seeds (chia, flax), olive oil (in moderation).
- **Fiber-Rich Foods:** Essential for gut health and detoxification (found in fruits, vegetables, whole grains, legumes).

Foods to Strictly Avoid:

- **Alcohol:** This is paramount. Even small amounts can be damaging to a compromised liver. Complete abstinence is highly recommended.
- **Highly Processed Foods:** Pre-packaged meals, fast food, processed meats (sausages, bacon).
- **Sugary Drinks & Added Sugars:** Sodas, fruit juices with added sugar, pastries, candies. Contributes to fatty liver.
- **Excessive Saturated & Trans Fats:** Fried foods, butter, fatty cuts of red meat, some baked goods.
- **High-Sodium Foods:** Canned goods, processed snacks, deli meats.

Hydration Guidelines:

- **Water:** Drink plenty of water throughout the day (aim for 8-10 glasses or as advised by your doctor).
- **Herbal Teas:** Unsweetened herbal teas can be a good addition.
- **Limit:** Sugary beverages, artificial sweeteners.



LIFESTYLE MODIFICATIONS:

Exercise Recommendations:

- **Moderate Intensity:** Aim for at least 150 minutes per week (e.g., 30 minutes, 5 times a week).
- **Mix of Cardio & Strength:** Include brisk walking, jogging, cycling, swimming, and incorporate strength training 2-3 times per week.
- **Consistency:** Regular activity helps manage weight, reduce liver fat, and improve insulin sensitivity.

Sleep Hygiene:

- **Consistent Schedule:** Go to bed and wake up at roughly the same time each day, even on weekends.
- **Optimal Environment:** Ensure your bedroom is dark, quiet, and cool.
- **Avoid Screens:** Limit exposure to electronics (phones, tablets, TVs) at least an hour before bedtime.

Stress Management:

- **Mindfulness & Meditation:** Practice deep breathing, meditation, or yoga.
- **Hobbies & Interests:** Engage in activities you enjoy to relax and de-stress.
- **Social Connection:** Maintain strong relationships with family and friends.
- **Time Management:** Prioritize tasks and learn to say no to avoid feeling overwhelmed.

Habits to Quit:

- **Smoking:** Damages blood vessels and contributes to liver disease progression.
- **Alcohol Consumption:** As mentioned, critical for liver health.
- **Illicit Drug Use:** Can directly harm the liver and overall health.

CRITICAL FACTORS:

Key Liver Health Indicators to Monitor (with your doctor):

- **Liver Function Tests (LFTs):** Alanine aminotransferase (ALT), Aspartate aminotransferase (AST), Alkaline phosphatase (ALP), Bilirubin.
- **Albumin:** A protein made by the liver; low levels can indicate liver damage.
- **INR (International Normalized Ratio):** Measures blood clotting time, affected by liver's ability to produce clotting factors.
- **Imaging Studies:** Ultrasound, FibroScan (a non-invasive test to assess liver stiffness/fibrosis), MRI.
- **Blood Glucose & Lipid Levels:** High blood sugar and cholesterol can contribute to fatty liver.

Warning Signs to Watch For (seek medical attention if experienced):

- **Jaundice:** Yellowing of the skin or eyes.
- **Dark Urine:** Urine that appears unusually dark (tea-colored).
- **Pale or Clay-Colored Stools:** Stools that lack their usual brown color.
- **Easy Bruising or Bleeding:** Due to impaired clotting factor production.
- **Severe or Persistent Fatigue:** Unexplained and overwhelming tiredness.
- **Unexplained Weight Loss.**
- **Abdominal Pain or Swelling:** Especially in the upper right quadrant, or a noticeable increase in belly size (ascites).
- **Confusion or Changes in Mental State:** Difficulty concentrating, memory problems (hepatic encephalopathy).
- **Persistent Nausea, Vomiting, or Loss of Appetite.**
- **Itchy Skin:** Without a rash.

GENERAL GUIDANCE:

Medication Precautions:

- **Always Inform Your Doctor:** When prescribed any medication, ensure your doctor is aware of your liver disease risk.
- **Over-the-Counter (OTC) Medications:** Be extremely cautious. Many common OTC drugs (e.g., acetaminophen/paracetamol in large doses, NSAIDs like ibuprofen) can be harmful to the liver. Always read labels and consult your pharmacist or doctor.
- **Avoid Self-Medication:** Do not start any new medications without medical advice.

Supplements to Consider/Avoid:

- **Consult Your Doctor:** Before taking ANY supplements, including vitamins, minerals, or herbal remedies. Some can be toxic to the liver or interact with medications.
- **Caution with Specifics:**
 - **Vitamin A:** High doses can be hepatotoxic (liver damaging).
 - **Iron Supplements:** Only take if prescribed, as excess iron can damage the liver.
 - **Herbal Supplements:** Many popular herbs (e.g., kava, comfrey, chaparral, high doses of green tea extract) have been linked to liver injury. Avoid unless explicitly approved by your doctor.
 - **Vitamin D:** Often recommended, but check levels with your doctor.
 - **Milk Thistle (Silybum marianum):** While widely used for liver health, its efficacy is debated, and it can interact with some medications. Discuss with your doctor before use.

By actively participating in these recommendations and maintaining open communication with your healthcare team, you are taking the best possible steps to manage and improve your liver health. Your commitment to these changes is your greatest asset.